

ХАКАТОН AVITO · КЕЙС «AI REVIEWER»

Avito AI Reviewer

Помощник ревьюера и координатора образовательных программ. Полная документация решения: процесс, архитектура, инструкции агента, защита данных, метрики, ограничения.

Локальный контур. Единственный внешний адрес в рантайме — модель на `localhost`. Персональные данные не покидают машину.

Итоговое решение — за человеком. Балл предварительный и публикуется студенту только после подтверждения ревьюером.

121 тест

проходят за 7 секунд

63 файла примеров

из репозитория кейса читаются без ошибок

0 внешних вызовов

в рантайме, кроме локальной модели

45 секунд

предварительное ревью одной работы

Содержание

1. Описание проекта

2. Установка, запуск и состав

3. Руководство пользователя и сценарий показа

4. Как добавить своё домашнее задание

5. Текущий и целевой процесс

6. Архитектура

7. Инструкции агента

8. Защита данных и обработка ошибок моделей

9. Метрики качества и эффекта

10. Стоимость эксплуатации

11. Масштабирование

12. Конфигурация

13. Честные ограничения

14. Сценарий демонстрации

Описание проекта

PROJECT.md

Обязательный артефакт сдачи. Структура: аннотация, проблематика, постановка задачи, описание технического решения, полученные результаты, дальнейшие планы, команда и распределение задач.

Аннотация

Проверка домашних заданий — самый ресурсоёмкий процесс в образовательных программах Авито. Работы поступают через GitHub, Stepik и Google Docs, ревьюеры оценивают их по критериям, а результаты и статусы переносят руками в Google Sheets. Эксперты тратят на проверку 5–10 часов в неделю, методисты — до 10 часов в неделю на поддержку процесса. С ростом программ нагрузка растёт линейно, а вместе с ней — число ошибок, пропущенных дедлайнов и задержек обратной связи.

Мы разработали MVP AI-помощника ревьюера и координатора. Система получает условие задания и решение студента, извлекает критерии оценивания из условия, проводит предварительное ревью и передаёт ревьюеру результат: баллы по каждому критерию, формальные нарушения, признаки использования генеративного ИИ и рекомендации. Координатор распределяет работы с учётом компетенций и нагрузки, контролирует сроки и получает аналитику. Итоговое решение всегда принимает человек.

Ключевое техническое решение — **проверка доказательств кодом**. Документ режется на пронумерованные блоки, и модель обязана подкрепить каждый балл дословной цитатой с указанием блока. Затем код ищет эту цитату в указанном блоке нечётким сравнением. Схема, которую заполняет модель, содержит только номер блока и текст цитаты — статус «подтверждено» проставляет код, поэтому модель физически не может объявить свою проверку пройденной. Не нашлась цитата — критерий помечается как неподтверждённый и поднимает приоритет ручной проверки, но балл при этом не снижается: техническая неудача сопоставления не доказывает невыполнение критерия.

Второе решение — **разделение детерминированного и вероятностного слоёв**. Объём, шрифт, кегль, межстрочный интервал, формат, сроки, штрафы, схожесть текстов и распределение нагрузки считает обычный код: на этих задачах он точен, объясним построчно и полностью воспроизводим. Модель вызывается только там, где нужна семантика — при оценке содержания по критериям. У формальных проверок четыре статуса вместо двух: «не определено» — полноправный ответ, потому что у реальных работ часть свойств не задана нигде, и трактовать отсутствие данных как нарушение значило бы штрафовать корректную работу.

Третье решение — **структурное отделение сигнала генеративного ИИ от оценки**. Кейс запрещает генеративной проверке автоматически влиять на балл студента. Мы не полагаемся на настройку: считаются два независимых числа разными формулами из разных слагаемых. Предварительный балл — сумма по критериям минус штрафы. Индекс приоритета ручной проверки — взвешенная сумма флагов риска. Сигнал детектора входит только во второе, ни в одном состоянии тумблера не касаясь балла. Сам сигнал — ансамбль из четырёх независимых признаков, каждый из которых деградирует явно: если у работы нет метаданных, сигнал по метаданным помечается «недоступен», а не «признаков не найдено», и итоговая уверенность снижается.

Решение работает **полностью офлайн**. Единственный внешний адрес — локальная Ollama с моделью `qwen3.5:9b`; всё остальное блокируется стражем и попадает в аудит, а интерфейс показывает счётчик внешних вызовов из фактически перехваченных обращений. Это не удобство, а требование: в метаданных реальных работ лежат настоящие ФИО авторов, и кейс запрещает передачу персональных данных в незащищённые сервисы. Псевдонимизация выполняется отдельным обязательным шагом до любого обращения к модели — если он не отработал, вызов не происходит вовсе.

На трёх предоставленных вариантах выполнения задания система устойчиво отделяет слабую работу от обеих сильных: разрыв 1,5–1,8 балла во всех трёх прогонах (6,5 против 8,3 и 8,0). Среднюю и хорошую работы она **не различает** — разрыв 0,3 балла, и причина локализована до одного критерия оформления, где у хорошей работы объективно нет ни таблиц, ни заголовочных стилей; по остальным четырём критериям их баллы совпадают. Разброс баллов между прогонами — **ноль**, подтверждаемость цитат — 88 %. Полное ревью одной работы занимает около 45 секунд на локальной GPU и не стоит ничего в переменных затратах. Число ручных действий на работу сокращается с 11 до 3 — измерено журналом фактических действий, а не расчётом. Для помощника ревьюера устойчивое выделение слабых работ ценнее различения двух сильных: именно слабые требуют внимания человека.

Проблематика

Авито Образование ведёт программы по продуктовому менеджменту, Data Science, аналитике данных и программированию. Участники выполняют задания и получают обратную связь от специалистов компании и приглашённых экспертов.

Текущий процесс состоит из семи шагов, и на одну работу приходится **11 ручных действий** — детальный разбор в `docs/as-is-to-be.md`.

Ключевые проблемы:

Проблема	Следствие
Работа сразу в нескольких системах: условие в Stepik, решение в Google Docs, результат в Google Sheets	переключение контекста, потерянные ссылки, расхождение данных
Ручной перенос баллов и комментариев в таблицу	опечатки, пропущенные строки
Ручной контроль сроков и просрочек	пропущенные дедлайны, задержки обратной связи
Распределение «поровну», без учёта ёмкости и компетенций	один ревьюер перегружен, другой простаивает
Разная трактовка критериев ревьюерами	одинаковые работы получают разные баллы
Балл выставлен, но не видно, на какой фрагмент работы он опирается	спорные случаи невозможно разобрать, студент не понимает оценку
Нагрузка растёт линейно с числом студентов	процесс не масштабируется

Постановка задачи

Разработать MVP AI-помощника ревьюера и координатора: на одном сквозном сценарии показать, как решение сокращает ручную работу, повышает прозрачность проверки и помогает соблюдать сроки.

Сквозной сценарий: направление `product_fraud`, ДЗ «Карта рисков продукта». Условие — `.pdf` (печать страницы Stepik), три решения — `.docx` (слабое, среднее, хорошее). Канал сдачи — Stepik.

Обязательные функции

Требование	Реализация
Распределять ДЗ между ревьюерами с учётом объёма курса и нагрузки	<code>app/workload/allocator.py</code> — венгерский алгоритм
Проводить предварительное ревью по критериям, формировать результат и рекомендации ревьюеру	<code>app/agent/</code> , <code>app/formal/</code>

Требование	Реализация
Выявлять признаки генеративного ИИ с уровнем уверенности и основаниями	app/detect/ai_text.py

Дополнительные функции (требовалось не менее трёх, реализовано семь): уведомления о дедлайнах · расчёт штрафа за просрочку по правилу из условия · аналитика прогресса и качества проверки · выявление дублирования и заимствований · персонализированная обратная связь студенту · выявление типовых ошибок потока для улучшения материалов · детекция и защита персональных данных.

Сверх обязательного минимума: повторная сдача с версиями и сравнением по критериям, профиль студента с динамикой и радаром сильных сторон, пресеты формулы приоритета, вкладка «Качество» с результатами бенчмарка внутри интерфейса.

Ограничения, принятые как проектные

- AI не заменяет ревьюера: итоговая оценка, спорные случаи и решения, влияющие на студента, подтверждаются человеком;
- сигнал о генеративном ИИ не является доказательством нарушения и не влияет на оценку автоматически;
- персональные данные, внутренние материалы и код не передаются в незащищённые сервисы;
- используются только реально существующие и доступные модели и сервисы;
- стоимость эксплуатации обоснована.

Описание технического решения

Архитектура

Один процесс `uvicorn` обслуживает API, поток событий и собранный интерфейс. Ни Docker, ни Redis, ни брокера, ни отдельного сервера БД: чем меньше движущихся частей, тем меньше зон отказа. Полная схема и обоснование стека — `docs/architecture.md`.



Пайплайн ревью

Список шагов, где дешёвое и надёжное считается раньше дорогого и вероятностного:

1. **формальные проверки** — без модели;

- 2. **маскирование ПДн** — обязательный шаг, до любого обращения к модели;
- 3. **оценка по критериям** — отдельный вызов на каждый критерий, затем проверка цитат кодом;
- 4. **признаки генеративного ИИ** — ансамбль сигналов;
- 5. **агрегация** — балл и индекс приоритета отдельно;
- 6. **обратная связь студенту** — текст, который правит ревьюер.

Упавший необязательный шаг пишется в трассу, и ревью продолжается. Обязательный останавливает пайплайн, но результат всё равно возвращается — с трассой, пометкой «проверить вручную» и повышенным приоритетом. Каскадных отказов нет по построению, и ревьюер всегда видит, что именно система успела проверить.

Почему отдельный вызов на каждый критерий

При оценке всех критериев одним запросом они «слипаются»: впечатление от сильного раздела переносится на слабый, и баллы сглаживаются. Отдельный вызов ещё и локализует отказ — упавший критерий не уносит остальные.

Модель и её настройка

`qwen3.5:9b Q4_K_M` через Ollama — крупнейшая модель, которая помещается на GPU 16 ГБ целиком вместе с контекстом 32k (9,9 ГБ). При выгрузке части слоёв в RAM время ревью выросло бы в разы.

Три настройки оказались критическими, и каждая закрыта тестом:

Настройка	Что происходит без неё
<code>LLM_REASONING_EFFORT=none</code>	<code>qwen3.5</code> — рассуждающая модель: вывод уходит в поле <code>thinking</code> , <code>content</code> остаётся пустым, генерация не завершается. Тривиальный запрос уходил в таймаут 180 с
<code>num_ctx 32768</code> в <code>Modelfile</code>	умолчание Ollama — 4096, а OpenAI-совместимый <code>/v1</code> не принимает <code>options</code> : хвост работы молча теряется
погашенные штрафы сэмплирования	базовая модель приносит <code>presence_penalty 1.5</code> ; при <code>temperature=0</code> штрафы всё равно правят logits до <code>argmax</code> , а структурный JSON состоит из повторяющихся токенов

Отдельная находка: Ollama принимает `response_format` и **молча игнорирует его** — проверено схемой с необычными именами полей. Поэтому схема дублируется в текст промпта, а фактическим принуждением структуры служит валидация Pydantic с ремонтным ретраем.

Провайдеро-независимость

Единственное место, знающее про модель, — `app/llm/client.py`; единственное место, знающее про Ollama, — `scripts/ollama_setup.ps1`. Весь код работает через OpenAI SDK против OpenAI-совместимого API, поэтому смена провайдера — правка `.env`. Готовые профили: локальная Ollama, собственный vLLM в контуре заказчика, облачный OpenAI. Профиль облака не заработает, пока его хост не добавлен в `ALLOWED_HOSTS` осознанно.

Защита данных

Подробнее — `docs/security.md`.

- **Офлайн-контур**: каждое обращение проходит через страж; хост вне `ALLOWED_HOSTS` роняет вызов до его выполнения.
- **ПДн-щит**: регулярные выражения (почта, телефон, мессенджер, ссылка, ИНН, СНИЛС, паспорт, карта с проверкой Луна, дата рождения) плюс NER `natasha` для имён, плюс отдельный проход по **метаданным документа** — ФИО автора лежит в `docProps`, а не в тексте. Замены обратимы: студент видит своё имя, а не псевдоним.

- Номер карты проверяется алгоритмом Луна, иначе под маску уходили бы суммы в расчёте ROI — содержательная часть работы.
- Найденное NER имя дополнительно разбирается `NamesExtractor`. Без этого фильтра под маску уходило название продукта, и модель видела `ЛИЦО_1` там, где студент описывал продукт, — а именно это она и оценивает. Точность счита здесь важна не меньше полноты: и пропуск ПДн, и лишняя маскировка портят результат, просто по-разному.
- Маскирование видно в интерфейсе: четвёртый слой просмотрщика подсвечивает в документе ровно те фрагменты, которые заменяются псевдонимами, — и считает их тот же код, что маскирует. Собственная эвристика для показа рисовала бы одно, а в модель уходило бы другое.
- Каталог `data/` целиком исключён из репозитория. В репозитории нет персональных данных: демо-пользователи синтетические.

Полученные результаты

Качество предварительного ревью

Три предоставленных решения, три прогона. Отчёт — `data/bench_report.json`, разбор — `docs/metrics.md`.

Работа	Балл	Разброс между прогонами	Нарушений	Цитат подтверждено	Время
слабое	6,5 / 10	0	1	87 %	40 с
среднее	8,3 / 10	0	1	81 %	49 с
хорошее	8,0 / 10	0	0	95 %	47 с

Разбор по парам — чтобы агрегат не скрывал, где система ошибается:

Пара	Верно прогонов	Разрыв	Вывод
слабое < среднее	3 / 3	+1,8	разделены уверенно
слабое < хорошее	3 / 3	+1,5	разделены уверенно
среднее < хорошее	0 / 3	-0,3	не разделены

- **правильность ранжирования — 67 %** (две пары из трёх);
- **стабильность — разброс 0** (проверено и в CLI, и через приложение);
- подтверждаемость цитат — 88 %;
- отказов шагов пайплайна — 0.

Инверсию средней и хорошей работы даёт единственный критерий — «Качество оформления»: он требует таблиц и заголовков, а у хорошей работы их нет, её структура выражена нумерацией в тексте. По остальным четырём критериям средняя и хорошая получают **ровно одинаковый балл**. Подробный разбор — в `docs/metrics.md`.

Отдельно стоит один замер. Когда подсветку персональных данных вывели в интерфейс, обнаружилось, что NER относит к людям название продукта, и в модель на месте продукта уходит `ЛИЦО_1`. После исправления изменился **ровно один критерий ровно у одной работы** — «Корректность описания продукта» у средней, 0,8 → 1,0; остальные четырнадцать значений не сдвинулись ни на сотую, а хорошая работа послужила контролем: у неё название продукта под маску не попадало, и её баллы не изменились. Гипотеза подтвердилась ровно там, где предсказывала.

Формальный слой даёт **три разных** набора нарушений на трёх работах: у слабого — посторонний шрифт в таблицах, у среднего — 10 пт в основном тексте вместо 11, у хорошего — нарушений нет, а межстрочный интервал честно помечен «не определено».

Правило сроков и штрафов

Проверено сквозным прогоном через приложение — три работы в трёх состояниях:

Работа	Состояние	Балл
сдана до срока	on_time	5,2 → 5,2
сдана через 35 мин после мягкого срока	late_penalty	7,5 → 6,5 (–1 по правилу из условия)
сдана после жёсткого срока	late_zero	8,0 → 0,0

Пересчёт выполнен планировщиком в течение 3 секунд, уведомления доставлены студенту и ревьюеру.

Качество на трёх курсах — включая тот, где вышло плохо

Курс	Формат решений	Критериев	Ранжирование
Технический QA	.xlsx	5	100 %
Продуктовый фрод	.docx	5	67 %
Системный дизайн	.md	9	67 %

Качество различения зависит от того, о чём спрашивает рубрика. Там, где критерии оценивают качество («реалистичный расчёт ROI», «обоснованная приоритизация»), система различает работы. Там, где они спрашивают о наличии («построена диаграмма», «определены интерфейсы»), объёмная слабая работа неотличима от хорошей — по восьми критериям из девяти они получают одинаковый балл. Обещать равномерное качество на всех курсах было бы неправдой, и мы этого не делаем.

Проверка на всём наборе примеров кейса

Через приложение прогнан не только основной сценарий. Из репозитория кейса выгружены шесть каталогов — **63 файла** в форматах .docx, .pdf, .xlsx и .md. Все разбираются без ошибок; единственный файл с нулевым текстом (решение, сданное сканом) система отвергает явно, а не оценивает вслепую.

Отдельно проверено, что продукт **рабочий, а не сценарный**: новый методист заведён существующим методистом через интерфейс и его руками пройден весь путь на курсе, которого в системе не было, — ревьюер, три студента, задание по техническому QA, загрузка условия, утверждение рубрики, три решения в .xlsx, распределение, ревью, результат.

Работа по QA	Балл	Цитат подтверждено	Время
слабое	12,5 / 20	84 %	60 с
среднее	15,5 / 20	96 %	115 с
хорошее	17,3 / 20	71 %	78 с

На этом курсе все три пары упорядочены верно — в отличие от основного сценария, где средняя и хорошая работы не разделяются.

Распределение по нагрузке

На демонстрационном потоке: ревьюер, загруженный на 3 работы из 4, не получает новых; все три работы уходят свободному и компетентному; ревьюер без компетенции по направлению не получает ничего.

Эффект

Метрика (формулировки из критериев успеха кейса)	Было	Стало
Число ручных действий на работу	11	3 (≈ –73 %)

Метрика (формулировки из критериев успеха кейса)	Было	Стало
Время ревьюера на работу	15–25 мин	5–8 мин
Время машины на работу	—	45 с, в фоне
Перенос результатов в таблицу	3 действия руками	0 — выгрузка по кнопке
Количество просроченных проверок	не измерялось	считается автоматически

Число ручных действий не берётся из этой таблицы: каждое действие пользователя пишется в журнал вместе с числом заменённых операций, и вкладка «Аналитика» показывает фактическое отношение.

Стоимость

Переменные затраты — **ноль**: локальная модель на имеющейся GPU. Для сравнения, 1000 работ через облачный API уровня GPT-4o mini обошлись бы примерно в 5,5 тыс. рублей и вынесли бы персональные данные из контура, что кейс запрещает. Подробно — [docs/cost.md](#).

Отдельный результат — оптимизация: время ревью худшей работы сокращено с 292 с до 49 с (**в шесть раз**) без потери качества, исправлением конфигурации модели и схемы вывода, а не упрощением проверки.

Тесты

181 тест, 12 с (включая 8 smoke-тестов на живой модели). Шестнадцать настоящих дефектов найдены проверками, а не на защите:

- регулярное выражение для телефона не обрабатывало разделитель «пробел + скобка», и номер вида **+7 (999) 123-45-67** уходил в модель **незамаскированным**;
- `Subscriber` был обычным `@dataclass`, поэтому не был хешируемым, и подписка на события падала — внешне это выглядело как «интерфейс не обновляется»;
- SQLite отдаёт даты без часового пояса, и распределение работ отвечало ошибкой 500 после первого перезапуска приложения;
- счётчик псевдонимов перебирал **значения** карты замен вместо ключей и всегда оставался нулём: разные люди получали один и тот же `ЛИЦО_1`, а обратная подстановка возвращала студенту чужое имя;
- коммит в тике планировщика стоял под условием «изменилось состояние работы», а напоминания о сроке проверки и сводка методисту создаются именно в тиках без изменений: уведомление уходило по SSE, но откатывалось вместе с сессией — и отправлялось заново каждую секунду;
- NER относил к людям название продукта («Автотека») и заголовок таблицы («Верхнеуровневая CJM»), и в модель на месте продукта уходил `ЛИЦО_1` — при том, что работа оценивается в том числе по корректности описания продукта.

Ещё пять нашлись, когда мы прогнали через приложение **весь репозиторий примеров кейса**, а не только три работы основного сценария:

- решение, сданное **сканом** (PDF на 1,3 МБ, ноль символов текста), уходило в модель пустым документом — система выставяла балл работе, которую не видела;
- работа, которую автоматика проверить не смогла, получала **нулевой индекс приоритета** и падала в самый низ очереди ревьюера — ровно туда, где её никто не увидит;
- решения в `.xlsx` доходят до 148 тысяч символов, и промпт **молча обрезал сам сервер модели**: балл выставялся по случайному куску;
- лист `.xlsx` превращался в **один блок 5N на 19 тысяч символов** — проверка цитат в блоке такого размера находит почти любую строку, то есть перестаёт быть проверкой;
- регулярка максимумов баллов знала только формат условия по фроду, и на условии по QA **кросс-проверка суммы молча выключалась**: рубрика с ошибкой модели (21 балл при заявленных в условии 20) выглядела

нормальной.

Два дефекта показательны тем, как нашлись. Шестой — не тестом, а тем, что подсветку персональных данных вывели в интерфейс: пока маскирование было невидимым, ошибка выглядела как норма. Десятый — тем, что взяли чужой курс: на трёх работах основного сценария проблема не проявлялась вовсе.

Правка десятого дала измеримый эффект на курсе QA: ранжирование было нарушено (13,5 / 15,5 / 15,0 — средняя выше хорошей), стало верным (12,5 / 15,5 / 17,3), а время ревью упало с 277 до 60–115 секунд.

Тринадцатый и четырнадцатый нашлись, когда мы взяли технические задания. **Тринадцатый:** `FormalRequirements` держал пороги ДЗ по продуктовому фроду как значения по умолчанию, и каждый другой курс молча получал чужие требования — работа по системному дизайну, которую её условие разрешает сдавать в Markdown, теряла балл за «формат MD не входит в разрешённые (DOCX, PDF)». **Четырнадцатый:** аналитика складывала баллы заданий с разными максимумами (10, 20 и 6) и показывала «средний балл 11,3» — число, не означающее ничего.

Двенадцатый дефект нашёлся там же и бил в самое чувствительное место — в **главную метрику кейса**. Базовая линия ручных действий считалась по работам выбранного задания, а фактические действия — по всему журналу целиком. С одним курсом числа сходились; со вторым метрика показала «было 33, стало 45, сокращение –36 %», то есть заявила, что система увеличила ручную работу. Числитель и знаменатель теперь описывают одни и те же работы, и это закреплено тестами.

Ограничения

Полный список — `docs/limitations.md`. Главные:

- **схемы и изображения не анализируются** текстовой моделью; каждое изображение помечается «не оценено», и именно поэтому критерий приоритизации у всех трёх работ получает 1,5 из 2 — визуальную матрицу проверить нечем;
- **метаданные доступны не всегда:** у экспорта из Google Docs `docProps/app.xml` отсутствует, и сигнал по метаданным помечается «недоступен», а не «признаков нет»;
- **детектор генИИ вероятностный** и не является доказательством;
- **бенчмарк на трёх работах** одного задания — показатель направления, а не статистика;
- **аутентификации нет** — она вне объёма MVP, и об этом прямо сказано на экране входа.

Дальнейшие планы

Ближайший шаг — пилот на одном курсе. Требуется SSO организации и разметка ревьюерами нескольких десятков работ, чтобы метрика «согласие ревьюера с AI» стала статистически значимой.

Направление	Что делать	Требует переработки архитектуры
Мультимодальность	модель, видящая схемы CJM и матрицы рисков	нет — интерфейс уже помечает «изображение не оценено»
Аутентификация	SSO организации, права на уровне программы	нет
Другие каналы сдачи	GitHub Pull Request со сравнением версий, Google Docs по ссылке	нет — изолировано в загрузке
Технические курсы	читатель <code>.ipynb</code> , оценка кода	нет — новый читатель за тем же интерфейсом
Масштаб выше одной GPU	<code>LLM_BASE_URL</code> → внутренний vLLM с батчингом	нет — правка <code>.env</code>

Направление	Что делать	Требует переработки архитектуры
Рост потока	DB_URL → PostgreSQL	нет — SQLAlchemy уже async
Тонкий анализ типовых ошибок	эмбединги вместо частотной кластеризации	нет

Ни одно направление не требует переработки архитектуры: точки расширения описаны в docs/scalability.md , и добавление курса или формата не влечёт правок в чужих модулях.

Команда и распределение задач

Раздел заполняется составом команды перед сдачей. Контакты и персональные данные участников в репозиторий не выносятся — они указаны в личном кабинете участника.

Участник	Роль	Зона ответственности	Степень участия
—	AI Engineer	пайплайн ревью, LLM-слой, проверка доказательств, детектор генИИ, ПДн-щит	—
—	AI Engineer	ingest, формальный слой, распределение, сроки, API, интерфейс	—
—	AI Product	анализ процесса AS-IS, границы MVP, критерии успеха, метрики эффекта, сценарий демонстрации	—

Как продуктовые решения повлияли на техническую реализацию

Связь прямая и проверяемая — вот четыре примера:

1. **«Ревьюер должен понимать, откуда балл»** → механизм проверки цитат кодом и разделение схем EvidenceIn / Evidence , чтобы модель не могла объявить свою проверку пройденной.
2. **«Нельзя штрафовать за то, чего мы не знаем»** → четвёртый статус UNKNOWN у формальных проверок и правило в промпте, запрещающее трактовать неопределённость как нарушение.
3. **«Сигнал ИИ не должен влиять на оценку»** → два независимых числа, посчитанных разными формулами, вместо одного балла с настраиваемым коэффициентом.
4. **«Демонстрация не должна зависеть от сети»** → офлайн-контур со стражем и счётчиком фактических вызовов вместо обещания в презентации.

Установка, запуск и состав

README.md

Помощник ревьюера и координатора образовательных программ Авито. Получает условие домашнего задания и решения студентов, проводит предварительное ревью по критериям и выдаёт ревьюеру баллы с доказательствами, формальные нарушения, сигнал о признаках генеративного ИИ и рекомендации. Распределяет работы с учётом нагрузки, следит за сроками, считает аналитику.

Итоговое решение всегда за человеком.

Решение работает **полностью офлайн**: единственный внешний адрес — локальная Ollama на localhost:11434 . Персональные данные не покидают машину.

Что внутри

Разделение работ	слабая отделяется от обеих сильных устойчиво (разрыв 1,5–1,8 балла, 3/3 прогонов). Среднюю и хорошую система не различает — разрыв 0,3, и он весь в одном критерии из пяти
Воспроизводимость	разброс баллов между тремя прогонами — ноль
Доказательства	88 % цитат подтверждены кодом, а не моделью
Время ревью	≈ 45 с на работу, в фоне
Ручных действий	было 11 на работу, стало 3 (измерено по журналу: 33 → 10 на потоке)
Тесты	181, проходят за 7 с (включая 8 smoke-тестов на живой модели)
Разбор примеров кейса	шесть каталогов ДЗ читаются без ошибок; поддержан 31 формат — .docx , .pdf , .xlsx , .md , .ipynb , исходный код и .zip с репозиторием
Внешних вызовов в рантайме	0

Для команды: развернуть у себя

В репозиторий не попадают data/ (там работы студентов, база и выгруженные примеры), .env и web/node_modules . Всё остальное — включая собранный web/dist — коммитится, поэтому Node для запуска не нужен.

```
# 1. Зависимости и .env
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1

# 2. Примеры работ из репозитория кейса (нужен интернет)
powershell -ExecutionPolicy Bypass -File scripts\fetch_examples.ps1
python scripts\make_samples.py

# 3. Демо-данные. --no-extract не обращается к модели
python -m app.seed --reset --no-extract

# 4. Запуск
powershell -ExecutionPolicy Bypass -File scripts\run.ps1
```

Без локальной модели

Интерфейс работает целиком: вход, все три роли, загрузка работ, критерии, сроки, распределение, аналитика, выгрузка, приватность. В шапке будет гореть красное «модель недоступна» — это честный индикатор, а не поломка.

Недоступна ровно одна вещь — **запуск проверки моделью**: кнопка вернёт ошибку соединения. Чтобы посмотреть, как выглядит проверенная работа, нужна Ollama с моделью (`scripts\ollama_setup.ps1`) либо чужая база `data/app.db` , переданная в обход git.

Критерии без модели тоже не извлекутся из файла условия автоматически — но их можно ввести кнопкой «Написать текстом» или «Править вручную».

Установка

Требуется Windows, Python 3.13, [Ollama](#) и GPU с 12+ ГБ VRAM. Интернет нужен **только** на этом шаге.

```
powershell -ExecutionPolicy Bypass -File scripts\setup.ps1
```

Скрипт поставит зависимости, создаст `.env` , настроит производные модели Ollama, поставит Node (портативно, без прав администратора) и соберёт интерфейс, выгрузит примеры работ, наполнит базу и прогонит тесты.

Запуск

```
powershell -ExecutionPolicy Bypass -File scripts\run.ps1
```

Откроется `http://127.0.0.1:8000` . С этого момента интернет не нужен.

`-ExecutionPolicy Bypass` обязателен: у Windows PowerShell 5.1 (команда `powershell`) политика по умолчанию — `Restricted` , и без флага запуск падает с `UnauthorizedAccess` . Флаг действует только на этот процесс, прав администратора не требует и настройки системы не меняет.

Стартовая страница — форма входа: логин и пароль. Пароль у всех демонстрационных учётных записей общий и напечатан на самом экране, там же списки логинов по ролям.

Роль	Логин	Пароль	Что видит
Методист	sokolova.i	avito2026	создание заданий, участники, критерии, загрузка работ, сроки, распределение, формула приоритета, аналитика, выгрузка, приватность
Ревьюер	dyachenko.o	avito2026	очередь, проверка с доказательствами, признаки ИИ, подтверждение
Студент	lebedeva.a	avito2026	сдача работы, свои работы, сроки, обратная связь

Логин выводится из имени: фамилия латиницей плюс инициал. Полный список — на экране входа, блок «Демонстрационные доступы».

Чтобы работать сразу тремя ролями, откройте **три вкладки** и войдите в каждой по-своему: сессия хранится в `sessionStorage`, а он изолирован по вкладке. Сменить роль внутри вкладки можно по имени пользователя в шапке.

Пароль хранится хешем `pbkdf2_sha256` с индивидуальной солью. Это форма входа, а не рубеж обороны: нет подписанной сессии, срока её жизни и ограничения частоты попыток — контур защищён тем, что приложение не выходит в сеть. Общий пароль и подсказка на экране отключаются ключами `DEMO_PASSWORD` и `DEMO_SHOW_CREDENTIALS` в `.env`.

Руководство со скриншотами и сценарий показа жюри — `docs/guide.md`. Тайминг и таблица трассируемости требований — `docs/demo_script.md`. Как завести своё задание и сдать по нему работу — `docs/add-homework.md`.

Кто что делает

Действие	Методист	Ревьюер	Студент
Завести задание	✓	✓	—
Задать критерии (файл условия или текст)	✓	✓ до утверждения	—
Утвердить критерии	✓	—	—
Загрузить работу	✓ за студента	—	✓ свою
Работа со своей темой	—	—	✓
Запустить проверку моделью	✓ по всему заданию	✓ по своей работе	—
Подтвердить и отправить результат	—	✓	—

Кнопка запуска проверки у ревьюера — в карточке работы справа, «Проверить заново моделью». У методиста — «Проверить все» в блоке распределения и «Проверить» в строке таблицы работ. Проверка идёт в фоне, 45–120 секунд на работу, результат приходит уведомлением.

Примеры работ для проверки

`data/примеры_домашек/` — условия и решения восьми курсов, разложенные по папкам плоскими файлами: их удобно выбирать в форме загрузки. Собираются из выгрузки репозитория кейса командой `python scripts/make_samples.py`. Все 83 файла разбираются приложением без ошибок; подробности — в `data/примеры_домашек/ЧИТАТЬ.md`.

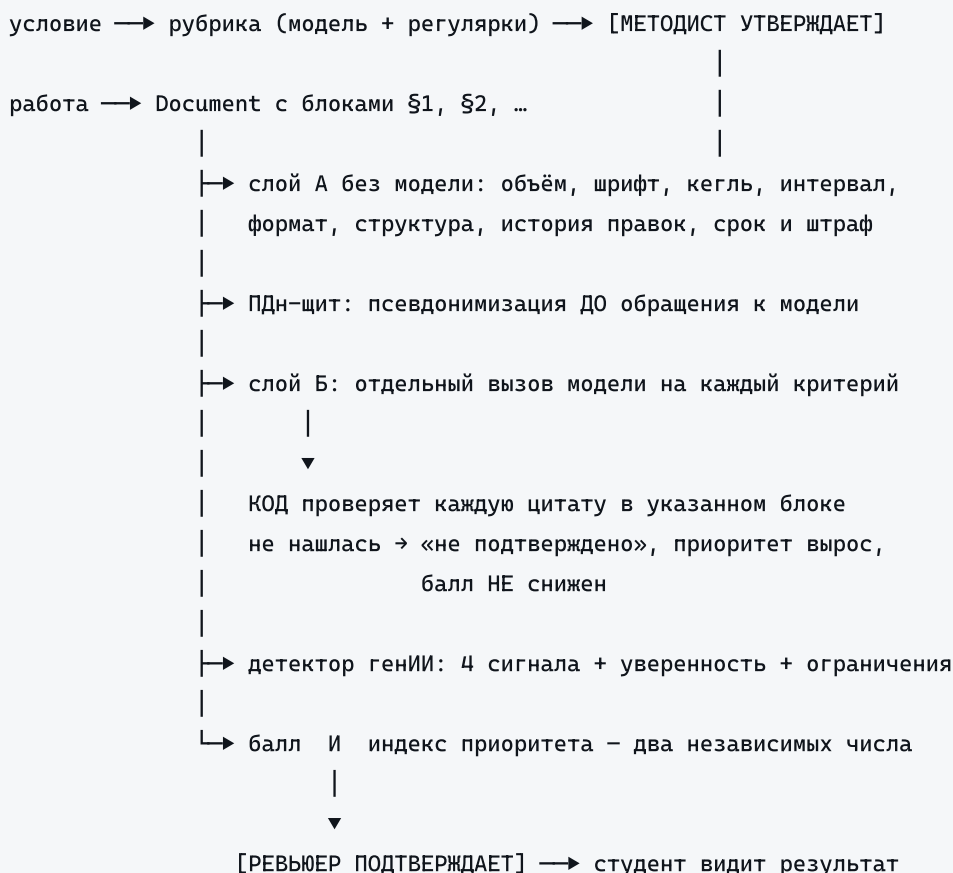
Другие задания из репозитория кейса

По умолчанию выгружаются два каталога примеров. Остальные — тем же скриптом, именами каталогов через запятую:

```
powershell -ExecutionPolicy Bypass -File scripts\fetch_examples.ps1 -Folders tech_QA,system_design,pro
```

Дальше всё делается в интерфейсе: «+ Новое задание» → загрузка условия → утверждение рубрики → «Загрузка работ». Все 63 файла шести каталогов (.docx , .pdf , .xlsx , .md) читаются приложением без ошибок.

Как это работает



Три вещи, которые отличают решение

1. Доказательства проверяются кодом, а не обещаются моделью. Документ режется на пронумерованные блоки. Модель обязана вернуть номер блока и дословную цитату — и только их: схема `EvidenceIn` содержит два поля. Статус «подтверждено» проставляет код, найдя цитату в указанном блоке нечётким сравнением. Модель физически не может объявить свою проверку пройденной.

2. Неопределённость — полноправный ответ. У формальных проверок четыре статуса, а не два. У одной из реальных работ межстрочный интервал не задан ни в параграфах, ни в стилях — честный ответ «не определено», а не «нарушение». Трактовать отсутствие данных как нарушение значило бы оштрафовать корректную работу.

3. Сигнал гЕНИИ отделён от балла структурно. Кейс запрещает генеративной проверке автоматически влиять на оценку. Поэтому считаются два независимых числа разными формулами: предварительный балл и индекс приоритета ручной проверки. Сигнал входит только во второе — ни в одном состоянии тумблера он не касается балла.

Проверка без интерфейса

```
# ревью одной работы в консоли
python -m app.cli review "data\examples\product_fraud\Product_Fraud_Д32_Решение хорошее.docx" \
    -c "data\examples\product_fraud\Product_Fraud_Д32_условия.pdf"

# бенчмарк на трёх работах
python -m app.bench.run_bench --runs 3

# тесты
python -m pytest -q
```

Документация

Файл	О чём
PROJECT.md	описание проекта — обязательный артефакт сдачи
docs/add-homework.md	как добавить своё ДЗ : от создания задания до проверенной работы, по шагам
docs/CASE.md	кейс и проверенные факты о данных
docs/PLAN.md	утверждённый план работ
docs/as-is-to-be.md	текущий процесс, проблемы, целевой процесс, расчёт эффекта
docs/architecture.md	компоненты, потоки данных, стек, точки расширения
docs/agent_instructions.md	инструкции агента и какую проблему решает каждое правило
docs/security.md	офлайн-контур, ПДн, ошибки моделей, роль человека
docs/cost.md	стоимость эксплуатации, локально против облака
docs/scalability.md	другой курс, формат, канал, масштаб
docs/configuration.md	.env, профили провайдеров, веса и тумблеры
docs/metrics.md	результаты бенчмарка и метрики эффекта
docs/limitations.md	честные ограничения и отклонения от плана
docs/demo_script.md	сценарий показа и трассируемость требований
docs/screenshots/	снимки интерфейса, тёмная и светлая тема; пересъёмка — <code>cd web && npm run shots</code> при запущенном приложении
docs/pdf/	документация и рассказ на защите одним файлом каждый — то, что отправляется жюри

Пересборка PDF

Оба документа собираются офлайн из тех же источников: markdown превращает в HTML `python-markdown`, печатает — Chromium, которым уже управляет Playwright.

```
cd web; npm run story-shots -- http://127.0.0.1:8000; cd ..
python scripts/build_docs_pdf.py
python scripts/build_story_pdf.py
```


Первый шаг снимает кадры для рассказа с работающего приложения — поэтому скриншоты в документе всегда соответствуют текущей версии, а не рисуются.

Стек

Python 3.13 · FastAPI · SQLite + SQLAlchemy async · OpenAI SDK → Ollama qwen3.5:9b · python-docx · pypdf · openpyxl · natasha · scipy · scikit-learn · React + TypeScript + Tailwind (Vite)

Обоснование выбора и того, чего сознательно нет (LangChain, Docker, Redis, Celery, внешние API), — в docs/architecture.md.

Структура

```
app/
  main.py config.py db.py models.py clock.py events.py queue.py seed.py cli.py
api/      deps.py routes.py
ingest/   document.py docx_reader.py pdf_reader.py xlsx_reader.py
          code_reader.py archive_reader.py loader.py
formal/   checks.py           - слой A, без модели
privacy/  pii.py audit.py      - ПДн-щит и офлайн-контур
llm/      client.py           - единственное место, знающее про модель
rubric/   models.py extract.py
agent/    pipeline.py prompts.py schemas.py evidence.py
scoring/  formula.py          - балл и приоритет отдельно
detect/   ai_text.py similarity.py
workload/ allocator.py
notify/   deadlines.py scheduler.py
bench/    run_bench.py
web/      src/ + dist/ (собран, коммитится - на демо Node не нужен)
          src/components/ui.tsx charts.tsx - графики рукописным SVG
          src/pages/ Coordinator Reviewer Student Manage Analytics
                  Scoring Quality Privacy StudentProfile Login
scripts/  setup.ps1 run.ps1 ollama_setup.ps1 fetch_examples.ps1 Modelfile.*
docs/
tests/    181 тест
```

Известные ловушки окружения

Каждая проверена на реальных данных и закрыта тестом.

Ловушка	Как проявляется	Как закрыто
qwen3.5 — рассуждающая модель	вывод уходит в поле thinking, content пустой, генерация не завершается	LLM_REASONING_EFFORT=none ; smoke-тест на непустой ответ
num_ctx у Ollama = 4096	хвост работы молча теряется; /v1 не принимает options	производная модель из Modelfile; smoke-тест читает фактическое значение из /api/show
/v1 игнорирует response_format	strict-схема не соблюдается	схема дублируется в промпт; принуждение — Pydantic с ремонтным ретраем

Ловушка	Как проявляется	Как закрыто
Шрифт и кегль наследуются	наивное чтение даёт <code>None</code> или неверное значение	разрешение цепочки <code>run</code> → стиль → <code>docDefaults</code> → тема
Нет <code>docProps/app.xml</code>	экспорт из Google Docs; объём и темп недоступны	«недоступно», а не «ноль»; уверенность сигнала снижается
SQLite не хранит часовой пояс	арифметика дат падает после перезапуска	общий <code>app/clock.py</code> , приведение на каждой границе с базой

Руководство пользователя и сценарий показа

docs/guide.md

Две части. Первая — что делает каждая роль и куда нажимать. Вторая — готовый сценарий показа жюри на трёх вкладках, по минутам.

Скриншоты сняты с рабочего приложения на демонстрационных данных.

Часть 0. Запуск

```
powershell -ExecutionPolicy Bypass -File scripts\run.ps1
```

Откроется `http://127.0.0.1:8000`. Остановка — **Ctrl + C** в том же окне.

Флаг `-ExecutionPolicy Bypass` обязателен: у Windows PowerShell 5.1 политика по умолчанию `Restricted`, и без флага скрипт не запустится.

Полезные ключи: `-Reseed` пересоздаёт демонстрационные данные с нуля, `-NoBrowser` не открывает вкладку автоматически.

Часть 1. Как пользоваться

Вход

!Форма входа

Логин и пароль. Роль определяется учётной записью — выбирать её на входе нельзя, иначе методистом стал бы любой желающий.

Кнопка «**Демонстрационные доступы**» раскрывает список: имя, роль, логин. Нажатие на строку подставляет логин в форму. Пароль у всех один и напечатан там же.

!Демонстрационные доступы

Роль	Логин	Пароль
Методист	sokolova.i	avito2026
Ревьюер (продуктовый фрод)	dyachenko.o	avito2026
Ревьюер (QA и системный дизайн)	gareev.t	avito2026
Студент	lebedeva.a	avito2026

Вкладка «**Регистрация**» рядом заводит новую учётную запись.

Три роли одновременно — три вкладки браузера. Сессия хранится в `sessionStorage`, а он изолирован по вкладке: инкогнито не нужно. Сменить роль внутри вкладки можно, нажав на имя пользователя в шапке.

Методист

!Экран методиста

Экран длинный, поэтому первым блоком идёт напоминание о порядке действий.

!Порядок действий

1. Критерии оценивания

Главный блок. Критерии — это то, за что выставляются баллы; без них проверять нечего.

!Критерии оценивания

Три способа задать, все равноправны:

- **«Загрузить файл условия»** — основной путь. Принимаются `.pdf`, `.docx`, `.md`, `.txt`. Система вытащит названия критериев, максимальные баллы, требования к оформлению, срок сдачи и правило штрафа. Числа берёт код регулярными выражениями, названия — модель.
- **«Написать текстом»** — когда файла условия нет. Вставьте условие как есть или напишите список «Критерий (2 балла)».
- **«Править вручную»** — точечная правка в JSON.

Дальше — **«Утвердить критерии»**. Пока не утверждены, проверка не запускается вовсе: ошибка в критериях исказила бы баллы всех работ разом. Любая правка сбрасывает утверждение.

2. Загрузка работ

!Загрузка работ

Выбрать студента и один или несколько файлов. Работы студент может сдавать и сам — со своей вкладки.

Принимается 31 расширение: документы, таблицы, ноутбуки, исходный код, `.zip` с репозиторием.

3. Сроки и штрафы

!Сроки и штрафы

Три момента времени: срок сдачи, окно опоздания (после него работа оценивается в ноль) и срок проверки.

«Режим демонстрации: 10 с / 40 с» сдвигает сроки так, чтобы переход в штрафную зону произошёл на глазах у зрителей, а не через сутки. После показа нажмите **«Вернуть обычные сроки»** — иначе данные останутся с просроченными сроками, и всё дальше будет выглядеть сломанным.

4. Распределение

!Распределение

«Распределить» раскладывает работы по ревьюерам венгерским алгоритмом: учитываются компетенции, текущая загрузка относительно ёмкости, срочность и трудоёмкость. График показывает нагрузку до и после.

Рядом — **«Проверить все (N)»**: массовый запуск проверки моделью по всему заданию.

5. Работы и выгрузка

!Таблица работ

Таблица всего потока: студент, файл, срок, статус, балл, приоритет, назначенный ревьюер. Кнопка **«Проверить»** в строке запускает проверку одной работы. Сверху — выгрузка в XLSX, CSV и JSON.

Ревьюер

!Очередь ревьюера

Слева очередь, отсортированная по тому, насколько вероятно ошиблась автоматика: слабые доказательства, признаки ИИ, балл на границе. Смотреть сверху вниз.

Кнопка «+ **Новое задание**» над очередью: ревьюер тоже может завести типовое ДЗ, которое студенты увидят при сдаче.

Карточка работы

!Шапка карточки

Справа вверху — предварительный балл и кнопка «**Проверить заново моделью**». Проверка идёт в фоне 45–120 секунд, результат приходит уведомлением. Ниже объяснение, почему работа оказалась наверху очереди.

Формальные проверки — слой без модели: объём, шрифт, кегль, интервал, формат, структура, история правок.

!Формальные проверки

У каждой проверки четыре состояния, а не два: **✓** соблюдено, **X** нарушено, **?** не определено (данных в файле нет) и **— не применимо** (условие такого требования не задаёт). Это важная деталь: требование, которого нет в условии, не может быть нарушено.

Оценка по критериям — балл по каждому критерию с цитатами из работы.

!Оценка по критериям

Цитаты проверены **кодом**: галочка означает, что цитата действительно найдена в указанном блоке работы. Не найдена — критерий помечается как неподтверждённый, и работа поднимается в очереди, но балл не снижается. Любой балл можно изменить руками.

Признаки генеративного ИИ — отдельный сигнал.

!Признаки ИИ

Четыре независимых источника, уровень уверенности, основания и ограничения проверки. Сигнал **не влияет на балл** — это требование кейса. Ревьюер может подтвердить его или отклонить.

Документ с подсветкой — сама работа с четырьмя слоями: доказательства, признаки ИИ, нарушения, персональные данные.

!Документ

Таблицы показываются таблицами, заголовки — заголовками, диаграммы и код — моноширинным блоком. Слева номер блока **§N** — на него ссылаются цитаты.

Итоговое решение — то, ради чего всё остальное.

!Итоговое решение

Балл, обратная связь студенту и кнопка «**Подтвердить и отправить**». До нажатия студент не видит ничего.

Студент

!Сдача работы

Выбрать задание, приложить файл, отправить. Если нужного задания в списке нет, есть пункт «**своя тема: указать название самому**» — карточка задания создастся автоматически, но критерии для неё должен будет описать ревьюер или методист.

Повторная сдача заменяет предыдущую версию, а не создаёт вторую работу: ревьюер увидит сравнение по критериям.

!Экран студента

Нижe — свои работы, сроки, оценка и обратная связь после подтверждения ревьюером. Плюс личный прогресс: динамика баллов и профиль по критериям.

Качество и приватность

Две вкладки в шапке, доступные методисту и ревьюеру.

!Эффект

«Качество» — сначала измеримые показатели эффекта: сколько ручных действий было и стало, время обработки работы, просроченные проверки. Это прямой ответ на критерий успеха кейса.

!Качество проверки

Нижe — результаты контрольного прогона: ранжирование, разброс между прогонами, доля подтверждённых цитат. Показана и **неудачная пара** тоже: цифра, которую прячут, перестаёт быть аргументом.

!Приватность

«Приватность» — счётчик внешних вызовов из аудита (а не декларация), журнал обращений и находки ПДн-щита.

Часть 2. Как показывать жюри

Восемь минут, три вкладки. Жюри просило показать, **как загружается ДЗ и как оно проверяется** — на это и построен сценарий.

Подготовка за пять минут до выхода

```
# 1. Проверить модель
powershell -ExecutionPolicy Bypass -File scripts\ollama_setup.ps1

# 2. Пересоздать данные (критерии основного ДЗ останутся НЕутверждёнными –
# так надо, их утверждение показывается живьём)
python -m app.seed --reset

# 3. Запустить
powershell -ExecutionPolicy Bypass -File scripts\run.ps1
```

Открыть **три вкладки** на `http://127.0.0.1:8000` и войти:

Вкладка	Логин	Роль
1	sokolova.i	методист
2	gareev.t	ревьюер
3	lebedeva.a	студент

Проверить перед началом: в шапке горит зелёное «**внешних вызовов: 0**» и «**модель локально**». Если нет — модель не поднялась, показывайте по заранее проверенным работам второго курса и скажите об этом честно.

Сценарий

0:00–0:40 · Вкладка 1. Что это. Показать шапку: три роли, счётчик внешних вызовов — ноль.

«Три роли, одна система. Наверху счётчик внешних вызовов: ноль. Это не декларация, а число из аудита — контур офлайн, единственный сетевой адрес это локальная модель на этой машине.»

0:40–1:40 · Вкладка 1. Откуда берутся критерии. Раскрыть «Критерии оценивания». Показать, что критерии, максимумы, требования к оформлению и срок извлечены из файла условия.

«Загружено условие домашнего задания. Пять критериев с максимумами 1-4-2-2-1, объём, шрифт, кегль, интервал, срок сдачи и правило штрафа — всё вытащено автоматически. Числа берёт код регулярками, названия — модель: цифры в условии однозначны, доверять здесь модели незачем.»

Показать рядом кнопку «Написать текстом».

«Если файла условия нет — требования можно просто написать текстом.»

1:40–2:00 · Вкладка 1. Человек утверждает. Нажать «Утвердить критерии».

«Это черновик. Пока критерии не утверждены, проверка **не запускается** — ошибка в критериях исказила бы баллы всех работ разом. Утверждает человек.»

2:00–2:40 · Вкладка 3. Студент сдаёт работу. Переключиться на вкладку студента, выбрать задание, приложить файл из `data/примеры_домашек/product_fraud/`, отправить.

«Работу сдаёт студент, сам. Файл может быть документом, таблицей, ноутбуком или репозиторием в архиве — тридцать одно расширение.»

Вернуться на вкладку 1: работа появилась в таблице без перезагрузки.

«У методиста она появилась сразу — приложение живое, без опроса сервера.»

2:40–3:20 · Вкладка 1. Распределение. Показать график нагрузки, нажать «Распределить».

«Павел уже загружен на три работы из четырёх, Ольга свободна, Тимур проверяет другие курсы. Венгерский алгоритм: это точный оптимум задачи о назначении, а не «раздать поровну».»

3:20–4:00 · Вкладка 1. Запуск проверки. Нажать «Проверить все».

«Проверку запускает человек — не система при загрузке. Работа уходит в фон, 45–120 секунд. Пока считает, покажу уже проверенную работу второго курса.»

4:00–6:00 · Вкладка 2. Ревьюер — главный экран. Здесь основное время.

1. Очередь: отсортирована по вероятности ошибки автоматике.
2. Формальные проверки: показать статус «не определено» и «не применимо».

«Четыре статуса, а не два. У этой работы межстрочный интервал не задан ни в параграфах, ни в стилях — честный ответ «не определено». Считать отсутствие данных нарушением значило бы оштрафовать корректную работу.»

3. Критерии: раскрыть один, показать цитаты с процентами.

«Модель обязана вернуть номер блока и дословную цитату — и только их. Статус «подтверждено» ставит **код**, найдя цитату в указанном блоке. Модель физически не может объявить свою проверку

пройденной.»

4. Нажать «показать» у цитаты — документ прокрутится к нужному блоку.

5. Признаки ИИ: показать уверенность, основания и ограничения.

«Сигнал не влияет на балл ни при каком положении настроек — это прямое требование кейса. Ревьюер может его отклонить.»

6:00–6:40 · Вкладка 2. Подтверждение. Изменить один балл, нажать **«Подтвердить и отправить»**.

«Решение принимает человек. До этой кнопки студент не видит ничего.»

6:40–7:10 · Вкладка 3. Что увидел студент. Обновить вкладку студента: появилась оценка и обратная связь.

«Студент получил балл и персональную обратную связь — не шаблон, а разбор его работы по критериям.»

7:10–7:40 · Вкладка 1. Вкладка «Качество».

«Показатели эффекта: ручных действий было столько, стало столько. Ниже — контрольный прогон на трёх работах разного уровня, включая пару, которую система **не** разделила. Мы её показываем, а не прячем.»

7:40–8:00 · Вкладка «Приватность».

«Ноль внешних вызовов за всё время показа. Персональные данные псевдонимизируются до обращения к модели — вот что нашёл щит.»

Если что-то пошло не так

Что случилось	Что делать
Модель не отвечает	Работы второго курса уже проверены заранее. Показывать их и сказать прямо: «проверка выполнена до демонстрации, чтобы не тратить ваше время на ожидание»
Проверка не запускается	Не утверждены критерии — посмотрите на значок рядом с выбором задания
Сроки выглядят просроченными	Нажать «Вернуть обычные сроки» в блоке «Сроки и штрафы»
Интерфейс не открылся	<code>python -m app.cli review <работа> -c <условие></code> — то же ревью в консоли
Данные испорчены показом	<code>python -m app.seed --reset</code> — 5 минут вместе с извлечением критериев

Заготовленные ответы

«Почему локальная модель, а не GPT-4?» Кейс запрещает передачу персональных данных и внутренних материалов во внешние сервисы. Локальная модель — не компромисс, а выполнение требования. Слой провайдеро-независимый: смена на облако — правка `.env`, не кода.

«Насколько модели можно доверять?» Ровно настолько, насколько её ответ проверяется кодом. 88 % цитат подтверждены программно. Балл предварительный, решение принимает человек.

«Что, если модель ошибётся?» Есть трасса выполнения: видно, какие шаги отработали, какие отказали. Отказавший шаг не роняет проверку, а поднимает работу в очереди ручного просмотра. Работу, которую не удалось прочитать, система отвергает, а не оценивает.

«Масштабируется ли на другие курсы?» Второй курс в демонстрации — технический QA, другой формат работ и другой максимум балла. Проверено на восьми каталогах примеров кейса: 83 файла разбираются без ошибок.

Как добавить своё домашнее задание

docs/add-homework.md

Пошаговая инструкция: от пустого экрана до проверенной работы. Всё делается через интерфейс, править код не нужно.

Роль для шагов 1–4 — **методист**. Шаг 5 выполняет студент или методист, шаг 6 — ревьюер.

Коротко

- 1. Методист: «+ Новое задание» → название, курс, направление.
- 2. Методист: «Критерии оценивания» → загрузить файл условия **или** написать требования текстом.
- 3. Методист: проверить критерии и нажать **«Утвердить критерии»**. Без этого проверка не запустится.
- 4. Методист: «Сроки и штрафы» → дедлайн (необязательно, если он был в условии).
- 5. Студент: «Сдать работу» → выбрать задание, приложить файл. Либо методист: «Загрузка работ» → выбрать студента и файлы.
- 6. Ревьюер: открыть работу в очереди → **«Проверить заново моделью»** → проверить баллы и цитаты → «Подтвердить и отправить». Методист может запустить проверку сразу по всему заданию — «Проверить все».

Дальше — то же самое подробно.

Шаг 1. Создать задание

Войдите методистом. Вверху справа — кнопка «+ Новое задание».

Поле	Что писать	Обязательно
Название	Как задание называется у студентов	да
Курс	Название курса целиком	да
Направление	Из списка: продуктовый фронт, QA, системный дизайн...	да
Канал сдачи	Откуда приходят работы (справочно)	нет

Направление — не украшение. По нему подбираются ревьюеры с подходящей компетенцией при автоматическом распределении, и по нему же система предупреждает, если загруженная работа похожа на работу другого курса.

Если нужного направления в списке нет, выберите ближайшее: на проверку это не влияет, влияет только на подбор ревьюера.

Шаг 2. Задать критерии оценивания

Критерии — это то, за что выставляются баллы. Без них проверять нечего.

Задать их можно **тремя способами**, они равноправны.

Способ 1. Загрузить файл условия

Кнопка **«Загрузить файл условия»** в блоке «Критерии оценивания». Принимаются `.pdf`, `.docx`, `.md`, `.txt`.

Система разберёт файл и вытащит:

- названия критериев и максимальные баллы за каждый;
- требования к оформлению (объём, шрифт, кегль, интервал, форматы файлов);
- срок сдачи и правило штрафа за опоздание, если они в условии есть.

Числа берёт код регулярными выражениями, названия критериев — модель. Так надёжнее: цифры в условии однозначны, доверять здесь модели незачем.

Способ 2. Написать требования текстом

Кнопка **«Написать текстом»**. Открывается поле, куда можно вставить условие целиком или набрать критерии списком. Формат свободный, но самый надёжный — строка на критерий с баллом в скобках:

```
Корректность описания продукта и ценностного предложения (1 балл)
Глубина и системность анализа рисков (4 балла)
Обоснованная приоритизация трёх критичных рисков (2 балла)
Реалистичный расчёт ROI митигации одного риска (2 балла)
Качество оформления (1 балл)
```

Нужно хотя бы 40 символов. Текст сохраняется как условие задания и проходит ровно тот же разбор, что и файл, — отдельной ветки нет.

Способ 3. Править вручную

Кнопка **«Править вручную»** под таблицей критериев открывает их в виде JSON. Способ для точечной правки: поменять вес, дописать признак, исправить максимум. Полностью набирать критерии здесь не нужно — для этого есть способы 1 и 2.

Что проверить перед утверждением

- **Сумма максимумов** совпадает с максимумом задания. Она показана под таблицей. Если условие говорит «максимум 10», а сумма даёт 12 — критерий разобран неверно.
- **Требования к оформлению** соответствуют условию. Если условие ничего не говорит про шрифт, в строке будет написано, что требований нет, и соответствующие проверки получают статус «не применимо». Это правильно: требование, которого нет в условии, не может быть нарушено.
- **Названия критериев** не повторяются. Повтор — признак того, что модель разбила один критерий надвое.

Шаг 3. Утвердить критерии

Кнопка **«Утвердить критерии»**.

Пока критерии не утверждены, проверка не запускается вовсе — ни вручную, ни массово. Это сделано намеренно: ошибка в критериях исказила бы баллы всех работ разом, и заметить это было бы поздно.

Любая правка критериев **сбрасывает** утверждение. Подтвердить нужно заново.

Шаг 4. Сроки и штрафы

Блок «Сроки и штрафы». Если срок был указан в условии, он уже заполнен — проверьте год: условия часто пишут дату без года, и система подставляет текущий, о чём предупреждает жёлтой строкой.

Задаются три момента времени:

- **срок сдачи** — после него работа считается опоздавшей;
- **окно опоздания** — сколько ещё принимаем со штрафом;
- **срок проверки** — до какого момента ревьюер должен закрыть работу.

Правило штрафа берётся из условия. Если условие штрафа не предусматривает, штрафа не будет.

Шаг 5. Загрузить работы

Студент сдаёт сам

Студент входит под своим логином, открывает блок **«Сдать работу»**, выбирает задание и прикладывает файл. Всё.

Работа со своей темой

Если задания в списке нет — преподаватель дал его на словах, карточку никто не заводил — студент выбирает в списке пункт **«своя тема: указать название самому»** и пишет название работы сам. Карточка задания создастся автоматически.

У такой работы **нет критериев**, и автоматическая проверка по ней не запустится, пока ревьюер или методист их не опишет: за что ставить баллы, кроме студента и преподавателя, никто не знает. Ревьюер увидит работу в очереди с прямым предложением задать критерии — блок «Критерии оценивания» открывается там же, в карточке.

Ревьюер заводит типовое ДЗ

Кнопка **«+ Новое задание»** есть и у ревьюера, над очередью. Заведённое задание сразу появляется у студентов в списке при сдаче. Утверждает критерии по-прежнему методист: право завести карточку и ответственность за оценку — разные вещи.

Если по этому заданию студент уже сдавал, новый файл уходит **следующей версией** той же работы: предыдущая сохраняется, ревьюер видит сравнение по критериям, а в очереди работа остаётся одна.

Методист загружает пачкой

Блок **«Загрузка работ»**: выбрать студента, выбрать один или несколько файлов. Для повторной сдачи указать, какую версию файл заменяет.

Какие форматы принимаются

31 расширение. Основные:

Что сдают	Форматы
Документы	.docx, .pdf, .md, .txt
Таблицы	.xlsx, .xlsm
Ноутбуки	.ipynb
Код	.py, .java, .go, .js, .ts, .sql, .sh и ещё десяток

Что сдают	Форматы
Репозиторий целиком	.zip

Полный список показан в самом блоке загрузки.

Скан без текстового слоя система отвергает, а не оценивает: PDF из одних картинок — это документ, которого текстовая модель не видит, и выставять по нему балл нельзя. Работа получает высший приоритет ручной проверки и отправляется человеку.

Шаг 6. Проверить

Кто запускает проверку моделью и где кнопка

Проверка **не запускается сама** — её запускает человек. Автоматический запуск при загрузке сознательно не сделан: модель занимает видеопамять и время, а решать, нужна ли машинная проверка этой работе, должен человек.

Кто	Где кнопка	Что делает
Ревьюер	карточка работы, справа вверху — « Проверить заново моделью »	проверяет одну работу; доступна всегда, в том числе чтобы перепроверить после правки критериев
Ревьюер	у непроверенной работы в центре карточки — « Запустить проверку »	то же самое, но заметнее, когда результата ещё нет
Методист	блок «Распределение по ревьюерам» — « Проверить все (N) »	ставит в очередь все работы задания разом
Методист	таблица «Работы», кнопка « Проверить » в строке	одна конкретная работа

Что происходит после нажатия: работа встаёт в очередь, проверка идёт в фоне 45–120 секунд в зависимости от размера, результат приходит уведомлением в колокольчик. Можно уйти на другую вкладку.

Кнопка не сработает в двух случаях, и оба объясняются на месте: критерии не утверждены и к работе не приложен файл. Работы без файла при массовом запуске пропускаются, о чём сообщается отдельно.

Дальше ревьюер смотрит:

- **формальные проверки** — объём, шрифт, кегль, интервал, формат; у каждой четыре статуса, включая «не определено» и «не применимо»;
- **оценку по критериям** — балл с цитатами из работы; цитаты помечены галочкой, если код нашёл их в указанном блоке, и волной, если не нашёл;
- **признаки генеративного ИИ** — отдельный сигнал, который **не влияет на балл** и который можно подтвердить или отклонить;
- **документ с подсветкой** — где именно в работе нашлись доказательства, нарушения и персональные данные.

Балл можно изменить у любого критерия. После этого — «**Подтвердить и отправить**». Только с этого момента студент видит оценку.

Частые вопросы

Ничего не разобралось из условия — что делать? Написать критерии текстом (способ 2). Разбор произвольного PDF — не гарантированная операция, и система об этом честно сообщает, а не подставляет выдуманные критерии.

Кнопка «Запустить проверку» выдаёт ошибку про файл. К работе не приложен файл. Такие строки бывают у демонстрационной нагрузки прошлого потока: они занимают место в очереди ревьюера, но содержимого у них нет. В очереди они помечены значком «без файла».

Проверка не запускается вообще. Скорее всего, не утверждены критерии — посмотрите на значок рядом с выбором задания. Второй вариант: не отвечает локальная модель, тогда в шапке горит красное «модель недоступна».

Сколько занимает проверка одной работы? От 45 секунд до двух минут, в зависимости от размера. Работа идёт в фоне: можно уйти на другую вкладку, результат придёт уведомлением.

Можно ли добавить новый курс, не трогая код? Да, именно это и описано выше. Второй курс в демонстрационных данных — технический QA — заведён целиком через интерфейс, без единой правки в коде.

Текущий и целевой процесс

docs/as-is-to-be.md

Пункты 1 и 2 постановки задачи: анализ процесса, ключевые проблемы, выбор этапов для автоматизации и проектирование целевого процесса.

1. Текущий процесс (AS-IS)

Процесс из семи шагов, описанный кейсодателем. В третьей колонке — сколько ручных действий приходится на **одну работу** на каждом шаге. Эти числа — базовая линия метрики «сокращение числа ручных действий», и они выведены здесь, а не взяты на глаз.

#	Шаг	Ручных действий на работу	Кто делает
1	Подготовка задания. Публикация условия и критериев, формирование таблицы проверки в Google Sheets, распределение студентов между ревьюерами	2 (создать строку в таблице, назначить ревьюера)	методист
2	Подготовка ревьюеров. Самостоятельное изучение условия и критериев, синхронизационная встреча	0 (разово на поток)	ревьюер
3	Выполнение и сдача. Сдача через Stepik, контроль сроков, напоминания	1 (напоминание о дедлайне)	методист
4	Первичное ревью. Открыть решение, сопоставить с условием и критериями, проверить полноту, определить балл, написать обратную связь	3 (открыть и прочитать, выставить баллы по критериям, написать фидбек)	ревьюер
5	Повторное ревью. Сравнение новой версии с предыдущей, проверка устранения замечаний	1 (сравнить версии)	ревьюер
6	Фиксация результатов. Перенос в Google Sheets балла, комментария, ссылки, статуса	3 (балл, комментарий, статус)	ревьюер
7	Контроль процесса. Отслеживание сданных и проверенных работ, просрочек, работ без оценки; связь при отклонениях	1 (проверка статуса)	методист
	Итого	11	

Цена процесса по данным кейса: эксперты — **5–10 часов в неделю**, методисты — **до 10 часов в неделю**.

2. Ключевые проблемы

Проблема	Где возникает	Следствие
Работа сразу в нескольких системах. Условие в Stepik, решение в Google Docs, результат в Google Sheets	шаги 4, 6	переключение контекста, потерянные ссылки, расхождение данных
Ручной перенос результатов. Балл и комментарий копируются руками	шаг 6	опечатки в баллах, пропущенные строки
Ручной контроль сроков. Дедлайны и просрочки отслеживаются глазами	шаги 3, 7	пропущенные дедлайны, задержки обратной связи

Проблема	Где возникает	Следствие
Распределение «поровну». Нагрузка не учитывает ни ёмкость ревьюера, ни его компетенции	шаг 1	перекос: один ревьюер перегружен, другой простаивает
Несогласованность оценок. Ревьюеры по-разному трактуют критерии	шаги 2, 4	одинаковые работы получают разные баллы
Непрозрачность обоснования. Балл выставлен, но не видно, на какой фрагмент работы он опирается	шаг 4	спорные случаи невозможно разобрать; студент не понимает оценку
Рост объёма. С числом студентов операционная нагрузка растёт линейно	весь процесс	процесс не масштабируется

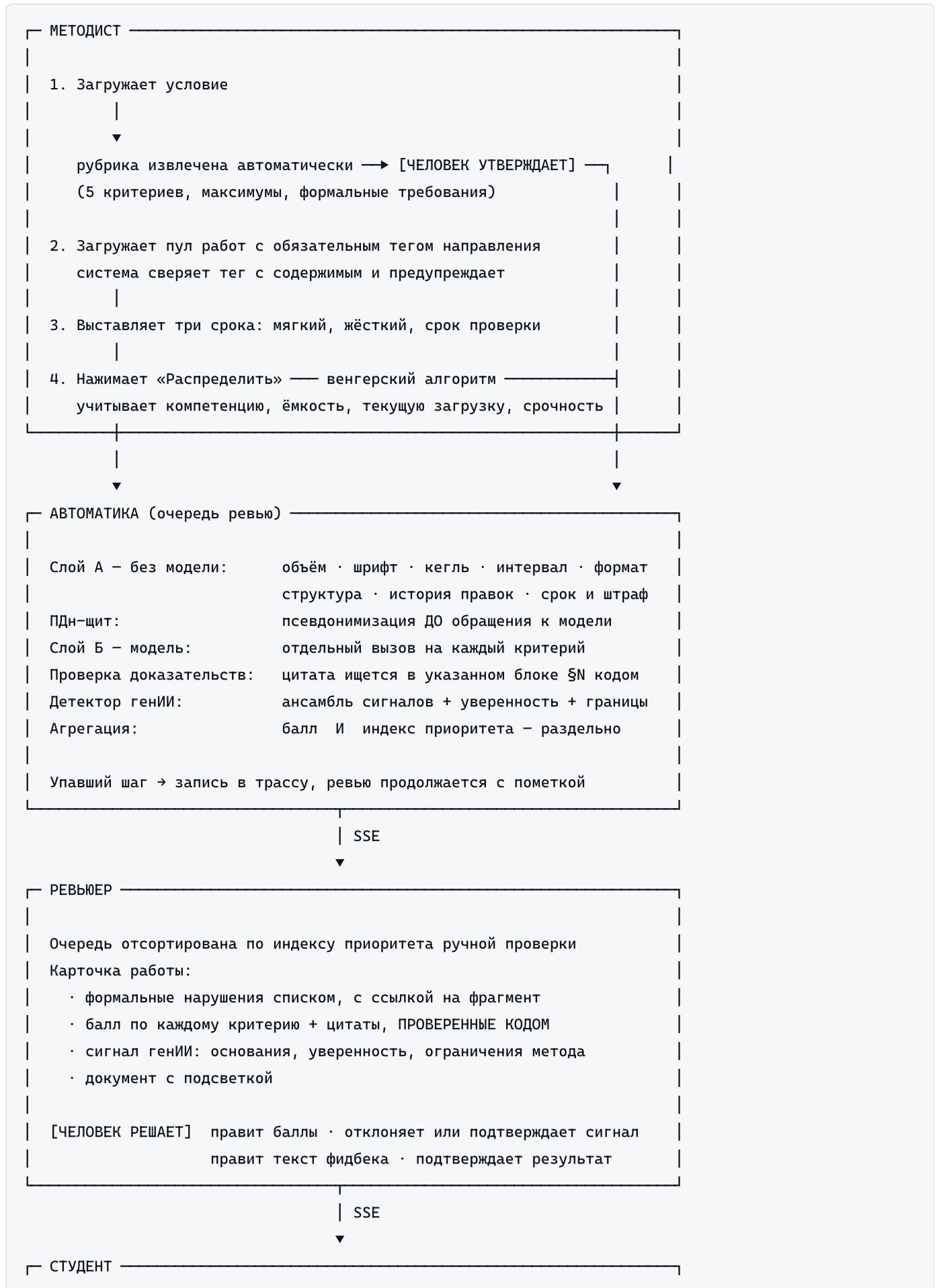
3. Какие этапы автоматизируются и почему именно они

Отбор шёл по двум признакам: доля ручного труда и допустимость автоматизации. Там, где решение влияет на студента, автоматизация останавливается на подготовке материала для человека.

Шаг	Решение	Обоснование
1. Подготовка задания	автоматизируем частично	критерии извлекаются из условия автоматически, но утверждает их человек : ошибка в рубрике исказила бы все баллы разом
1. Распределение	автоматизируем полностью	детерминированная задача о назначении, решается точно; ручной перенос остаётся доступен
2. Подготовка ревьюеров	не автоматизируем	это обучение людей, а не операция
3. Контроль сроков и напоминания	автоматизируем полностью	правило сроков и штрафа задано условием однозначно
4. Первичное ревью	автоматизируем как предварительное	предварительный балл с доказательствами и рекомендациями; итог подтверждает ревьюер
5. Повторное ревью	автоматизируем частично	сравнение версий и проверка устранения замечаний; вердикт за человеком
6. Фиксация результатов	автоматизируем полностью	выгрузка вместо ручного копирования; строчка в таблице появляется сама
7. Контроль процесса	автоматизируем полностью	дашборд вместо ручного просмотра таблицы

Что сознательно не автоматизируется (границы из кейса): итоговая оценка, спорные случаи, санкции по признакам генеративного ИИ.

4. Целевой процесс (TO-BE)



Видит балл и обратную связь — ТОЛЬКО после подтверждения человеком.
 Внутренние флаги (приоритет, сигнал ИИ, схожесть) не передаются
 вовсе: сервер не отдаёт их этой роли.

и параллельно
 сроки: 4 состояния, автопересчёт
 штрафа и уведомления по SSE
 аналитика: воронка, баллы,
 загрузка, типовые ошибки
 выгрузка: XLSX / CSV / MD / HTML

5. Как меняется число ручных действий

Шаг	Было	Стало	Что заменило
1. Подготовка задания	2	1	утверждение рубрики одним нажатием
1. Распределение	1 на работу	1 на весь поток	одно нажатие «Распределить»
3. Контроль сроков	1	0	планировщик и уведомления
4. Первичное ревью	3	1	подтверждение готового ревью
5. Повторное ревью	1	1	без изменений
6. Фиксация результатов	3	0	результат уже в системе, выгрузка по кнопке
7. Контроль процесса	1	0	дашборд обновляется сам
Итого на работу	11	3	

Расчётное сокращение — **около 73 %**. Приложение не полагается на этот расчёт: каждое действие пользователя пишется в журнал `ActionLog` вместе с числом заменённых ручных операций, и вкладка «Аналитика» показывает **фактическое** отношение, а не эту таблицу. Формулировки метрик взяты дословно из критериев успеха кейса.

6. Что остаётся за человеком

Список закрытый и проверяется кодом:

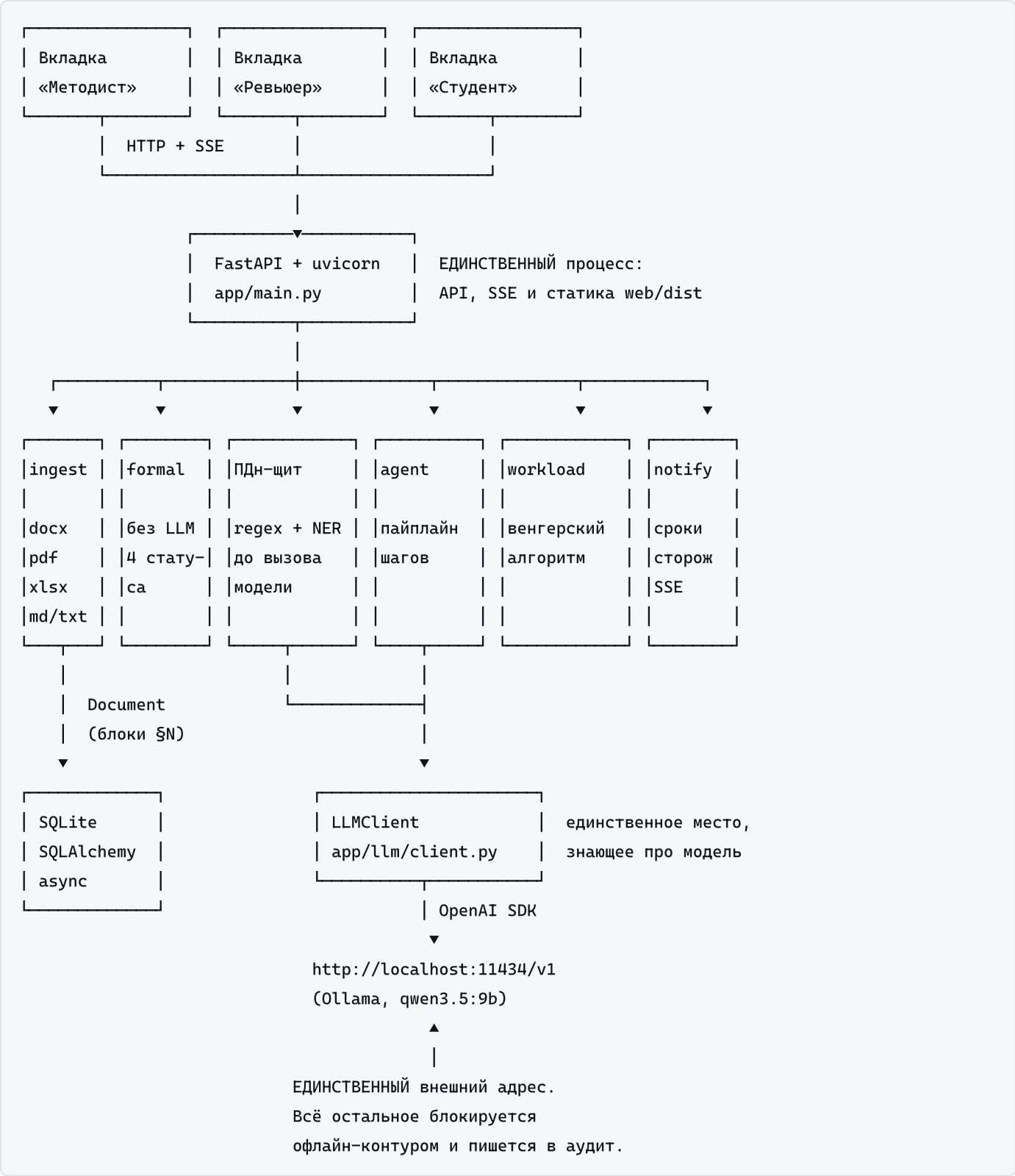
- **итоговая оценка** — публикуется студенту только после `POST /confirm`;
- **рубрика** — ревью не стартует, пока `rubric_approved` равно `false`;
- **сигнал генеративного ИИ** — вклад идёт в приоритет проверки, но **не в балл**; подтвердить или отклонить может только ревьюер;
- **непроверенная цитата** — балл не снижается автоматически: техническая неудача сопоставления не является доказательством невыполнения критерия;
- **спорные случаи** — работы с высоким индексом приоритета поднимаются в начало очереди именно для того, чтобы их посмотрел человек.

Архитектура

docs/architecture.md

Пункт 2 постановки задачи и пункт 7 (обоснование архитектуры, стека, моделей, интеграций).

Общая схема



Компоненты и их роли

Модуль	Роль	Ключевое решение
app/ingest/	.docx, .pdf, .xlsx, .md, .ipynb, исходный код и .zip → единый Document с блоками §N	Разрешение наследования стилей Word. Наивное чтение run.font.size возвращает None там, где значение унаследовано; таблицы читаются структурно, а не склеиваются в поток. Лист .xlsx режется на блоки по строкам: один блок на лист (19 тыс. символов на реальном решении) обесценивал проверку цитат
app/formal/	Слой A: проверки без модели	Четыре статуса вместо двух. UNKNOWN — полноправный ответ: у одной из реальных работ межстрочный интервал не задан нигде, и «нет данных» нельзя трактовать как нарушение
app/privacy/	Псевдонимизация до вызова модели + аудит вызовов	Маскируются и текст, и метаданные: ФИО автора лежит в docProps, а не в тексте. Найденное NER имя дополнительно разбирается NamesExtractor: без этого под маску уходило название продукта
app/llm/	Провайдеро-независимый клиент	Схема дублируется в промпт: Ollama принимает response_format и игнорирует его. Фактическое принуждение — валидация Pydantic с ремонтным ретраем
app/agent/	Пайплайн ревью как список шагов + проверка цитат	Отдельный вызов на критерий: при оценке всех сразу критерии «слипаются». Упавший шаг пишется в трассу, ревью продолжается
app/scoring/	Балл и индекс приоритета — два независимых числа	Кейс запрещает генеративной проверке влиять на оценку; разделение сделано структурно, а не на словах
app/detect/	Признаки генИИ, схожесть работ, типовые ошибки	Ансамбль сигналов, каждый деградирует явно. Схожесть — TF-IDF и шинглы, без моделей и сети
app/workload/	Распределение по нагрузке	linear_sum_assignment — точный оптимум, а не эвристика
app/notify/	Сроки, состояния, уведомления	Часы не подкручиваются: время реальное, двигаются сроки
app/queue.py	Очередь ревью	Синхронный пайплайн уносится в поток через asyncio.to_thread, иначе вызов модели блокирует event loop и SSE-прогресс не доходит
app/events.py	Одна SSE-шина на всё приложение	Адресация по роли и пользователю: студент не получает чужие внутренние флаги даже в теле ответа
app/clock.py	Единственный источник «сейчас» и приведение к UTC	SQLite не хранит часовой пояс; без общего приведения арифметика дат падает после первого перезапуска

Пайплайн ревью

Шаги выполняются по порядку: дешёвое и надёжное раньше дорогого и вероятностного.

#	Шаг	Обязательный	Что делает
1	читаемость документа	да	из скана без текстового слоя оценивать нечего
2	формальные проверки	нет	слой A, без модели
3	маскирование ПДн	да	без него обращаться к модели нельзя; здесь же работа обрезается по бюджету контекста
4	оценка по критериям	да	отдельный вызов на критерий + проверка цитат
5	признаки генИИ	нет	ансамбль сигналов

#	Шаг	Обязательный	Что делает
6	агрегация	нет	балл и приоритет отдельно
7	обратная связь студенту	нет	текст, который правит ревьюер

Деградация. Необязательный шаг при отказе пишется в трассу, и ревью идёт дальше. Обязательный останавливает пайплайн, но результат всё равно возвращается — с трассой, пометкой «проверить вручную» и повышенным индексом приоритета. Каскадных отказов нет по построению.

Антигаллюцинационный механизм

Главный технический аргумент решения.

1. Документ режется на пронумерованные блоки §1, §2, ...
Нумерация одна и та же в промпте и при проверке.

2. Модель обязана вернуть для каждого балла:

```
{ "block": 17, "quote": "дословный фрагмент" }
```


Схема EvidenceIn содержит ТОЛЬКО эти два поля.

3. КОД ищет цитату в указанном блоке нечётким сравнением
(`rapidfuzz.partial_ratio`, порог 82 %).

4. Итог:

`verified`

– найдена в указанном блоке

`wrong_block`

– найдена, но в другом блоке

`not_found`

– в работе такого текста нет

`no_block`

– указанного блока не существует

5. Ни одна цитата не подтвердилась → критерий помечается
``unverified``, индекс приоритета растёт, работа поднимается
в очереди. Балл при этом НЕ снижается: неудача сопоставления
не доказывает невыполнение критерия.

Почему нечёткое сравнение, а не точное: модель нормализует пробелы, меняет регистр и кавычки при переносе. Посимвольное равенство отбраковывало бы честные цитаты.

Почему статус не приходит от модели: наблюдалось, что при наличии поля `status` в схеме модель сама проставляет `"verified"` и `"similarity": 1.0`. Разделение типов `EvidenceIn` (вход) и `Evidence` (после проверки) делает это структурно невозможным.

Потоки данных

Загрузка работы

- файл → `_save_upload` → `sha256` → определение направления по содержимому

→ сверка с указанным тегом → предупреждение при расхождении

→ `Submission(status=uploaded)`

Ревью

```
POST /submissions/{id}/review
→ проверка rubric_approved (иначе 400)
→ queue.enqueue → таблица Job + asyncio.Queue
→ воркер: asyncio.to_thread(пайплайн)
    прогресс → SSE → браузер
→ результат в Submission.review (JSON)
→ SSE review_ready + уведомление ревьюеру
```

Наступление срока

```
секундный сторож → evaluate(сроки) → состояние изменилось?
→ пересчёт штрафа → пересчёт балла в сохранённом ревью
→ SSE deadline_changed + уведомления студенту и ревьюеру
```

Стек и обоснование

Слой	Выбор	Почему именно так
Backend	Python 3.13 + FastAPI + uvicorn	один процесс, async из коробки, уже установлено
БД	SQLite + SQLAlchemy 2.0 async	ноль инфраструктуры; результаты ревью в JSON-колонках, потому что структура эволюционирует, а миграции на этом сроке дороже гибкости. Единственный шаг миграции — аддитивное добавление недостающих колонок при старте: <code>create_all</code> создаёт только отсутствующие таблицы и на старой базе приложение падало бы
Очередь	<code>asyncio.Queue</code> + воркеры + таблица <code>Job</code>	без Redis и Celery. Таблица нужна не для распределённости, а для наблюдаемости: после перезапуска видно недопроверенные работы
События	SSE	поток односторонний, браузер переподключается сам
LLM	<code>openai</code> SDK → OpenAI-совместимый API	смена провайдера = правка <code>.env</code> . Профили в <code>configs/providers/</code>
Модель	<code>qwen3.5:9b</code> Q4_K_M через Ollama	9,9 ГБ с контекстом 32k целиком на GPU 16 ГБ; русский язык, структурный вывод
docx	<code>python-docx</code> + прямое чтение XML	библиотека не разрешает наследование стилей — приходится читать <code>styles.xml</code> и <code>theme1.xml</code> самим
pdf	<code>pypdf</code>	текстовый слой в условии есть, OCR не нужен
xlsx	<code>openpyxl</code>	второй курс и QA
NER	<code>natasha</code>	русские ПДн, CPU, без torch; при отсутствии деградирует до регулярок
Распределение	<code>scipy.optimize.linear_sum_assignment</code>	точный оптимум задачи о назначении
Схожесть	<code>scikit-learn</code> TF-IDF + шинглы	символьные n-граммы устойчивы к флексии русского; отдельная эмбединг-модель не нужна

Слой	Выбор	Почему именно так
Frontend	React + TypeScript + Tailwind, Vite	<code>web/dist</code> собран заранее и коммитится: на демонстрации Node не нужен
Графики	рукописный инлайновый SVG	гистограмма, столбцы по дням, линия динамики и радар по критериям не стоят зависимости, а контур офлайн — CDN недоступен. Цвета рядов заданы отдельно для светлой и тёмной темы и проверены на разделимость при дальтонизме

Чего сознательно нет: LangChain, Docker, WSL, Redis, Celery, внешних API, Telegram, SMTP, отдельной эмбединг-модели. Каждый из них добавил бы зону отказа, не решая задачи кейса.

Точки расширения

Добавление сущности не требует правок в чужих модулях.

Что добавить	Куда	Правки в других модулях
Новый формат файлов	читатель → <code>READERS</code> в <code>app/ingest/loader.py</code>	нет
Решение из многих файлов	<code>.zip</code> разбирается в один документ со сквозной нумерацией блоков	нет
Новая формальная проверка	функция с декоратором <code>@check</code> в <code>app/formal/checks.py</code>	нет
Новый сигнал генИИ	функция → список в <code>detect_ai_text</code>	нет
Новый шаг пайплайна	<code>Step(...)</code> в <code>STEPS</code>	нет
Новый компонент приоритета	поле в <code>PriorityWeights</code> + слагаемое в <code>aggregate</code>	нет
Новый курс или ДЗ	загрузка условия и утверждение рубрики через интерфейс	кода нет вовсе
Другой LLM-провайдер	<code>.env</code> или файл из <code>configs/providers/</code>	кода нет вовсе

Подробнее — `scalability.md`.

Инструкции агента

docs/agent_instructions.md

Пункт 7 постановки задачи требует обосновать инструкции агента. Здесь — каждый промпт, его роль, схема вывода и объяснение, **какую наблюдавшуюся проблему решает каждое правило**. Ни одно правило не написано «для стиля»: все добавлены после конкретного сбоя на реальных данных.

Исходники: `app/agent/prompts.py` , схемы — `app/agent/schemas.py` .

Общие принципы

- 1. **Модель вызывается только там, где нужна семантика.** Объём, шрифт, кегль, интервал, сроки, штрафы, схожесть текстов, распределение нагрузки считает детерминированный код. Модель оценивает содержание — и всё.
- 2. **Отдельный вызов на каждый критерий.** При оценке всех критериев одним запросом они «слипаются»: впечатление от сильного раздела переносится на слабый, и баллы сглаживаются. Отдельный вызов ещё и локализует отказ.
- 3. **Схема дублируется в текст промпта.** Ollama 0.24 принимает `response_format` и молча его игнорирует — проверено схемой с необычными именами полей. Фактическое принуждение структуры — валидация Pydantic с ремонтным ретраем.
- 4. **Воспроизводимость.** `temperature=0` , фиксированный `seed` , погашенные штрафы сэмплирования. Разброс баллов между прогонами — ноль (см. `metrics.md`).
- 5. **Ограничение длины ответа.** `max_tokens` плюс потолки на число цитат и их длину. Потолки **обрезают**, а не отвергают: полный ретрай из-за избыточного усердия модели — плохой размен.

1. Ревью по критерию

Роль: оценить одну работу по одному критерию рубрики. **Схема вывода:** `CriterionVerdict` .

score	float, 0 ... max_score критерия
verdict	краткий вывод, 1–2 предложения
evidence	до 6 объектов { block: int, quote: str }
gaps	до 5 строк: чего не хватает до полного балла
recommendation	рекомендация РЕВЬЮЕРУ, не студенту

Правила и что каждое лечит

#	Правило	Какую проблему решает
1	Оценивать строго по переданному критерию, не переносить впечатление	«слипание» критериев при общей оценке
2	Дословная цитата с номером блока §N ; маркеры §N проставлены системой, в работе их нет, в текст цитаты не включать	модель включала маркер в цитату, и проверка не находила её в блоке
3	Подтверждения нет — оставить evidence пустым и объяснить в gaps	выдумывание правдоподобных цитат
4	Балл — число от 0 до максимума критерия	значения выше максимума искажали сумму

#	Правило	Какую проблему решает
5	Дисциплина оценки: считать от идеала вниз; максимум только при выполнении всех признаков; заявленное, но не раскрытое требование считается невыполненным; не завышать «из вежливости»	без этого правила слабая работа получала 7,0 из 10 при верном порядке ранжирования. После добавления — 5,2, разделение работ выросло втрое
6	В <code>gaps</code> — конкретика, а не общие слова	«нужно доработать» бесполезно ревьюеру
7	<code>recommendation</code> адресована ревьюеру	иначе текст дублировал фидбек студенту
8	Схемы и изображения модель не видит — сказать об этом в <code>gaps</code>	оценка схемы наугад
9	Об оформлении судить только по блоку «УСТАНОВЛЕНО КОДОМ»	модель судила об оформлении по извлечённому тексту и снижала балл за «сплошной поток с номерами блоков» — то есть штрафовала студента за нашу собственную нумерацию
10	Статус «НЕ ОПРЕДЕЛЕНО» — это не нарушение, снижать за него балл нельзя	модель принимала «интервал не определён» за нарушение требования — ровно та ошибка, ради которой в формальном слое заведён отдельный статус
11	Только JSON-объект по схеме	markdown-ограждения и преамбулы вида «Вот результат:»

Блок «УСТАНОВЛЕНО КОДОМ»

Передаётся в каждый вызов. Содержит факты, которые модель не может установить по тексту, со **словесным статусом** каждой формальной проверки:

- формат файла: DOCX
 - заголовков: 0
 - элементов списков: 13
 - таблиц: 0
 - изображений и схем: 1 (их содержимое не анализировалось)
 - объём: 9077 символов, 1124 слов, 3 стр. по метаданным
 - шрифты: Arial, Cambria Math
 - кегли: 11 пт
 - межстрочный интервал: не определён
 - результаты формальной проверки:
 - [СОБЛЮДЕНО] Объём работы: 3 стр. при лимите 3.
 - [НЕ ОПРЕДЕЛЕНО – данных нет, это НЕ нарушение, балл за это не снижать]
- Межстрочный интервал: не задан ни в параграфах, ни в стилях...

Статус выписан словами, а не кодом `unknown`: наблюдалось, что при сыром сообщении модель всё равно трактовала неопределённость как нарушение.

2. Обратная связь студенту

Роль: превратить внутренний результат ревью в письмо студенту. **Вывод:** свободный текст (схема не нужна, текст всё равно правит ревьюер).

Правило	Зачем
По-русски, на «вы», доброжелательно	адресат — учащийся

Правило	Зачем
Начать с того, что получилось, — конкретно	иначе фидбек читается как отписка
Затем что улучшить: действия, а не оценки	«добавьте матрицу вероятность × влияние», не «слабый анализ»
Не называть баллы по критериям и не спорить с оценкой	итог подтверждает ревьюер, и он может отличаться от предварительного
Не упоминать внутренние флаги: признаки ИИ, схожесть работ, приоритет проверки	это внутренняя кухня проверки; сигнал ИИ не доказательство, и предъявлять его студенту нельзя
150–250 слов, без канцелярита	читаемость

После генерации выполняется **обратная подстановка ПДн**: студент видит своё имя, а не `ЛИЦО_1`.

3. Судья по признакам генеративного ИИ

Роль: оценить стилистические признаки машинной генерации. Один из четырёх сигналов ансамбля, вес 0,35.

Схема вывода:

likelihood	float 0...1
observations	наблюдаемые признаки
quotes	цитаты с указанием §N
alternative_explanations	что объясняет признаки без применения ИИ

Правило	Зачем
Не выносить обвинение, только перечислять признаки и уверенность	кейс прямо запрещает считать сигнал доказательством
Каждый признак — с цитатой	иначе вывод непроверяем
Деловой и учебный текст сам по себе шаблонен; шаблонность — слабый признак	иначе любая аккуратная работа помечается подозрительной
Обязательно назвать альтернативные объяснения	требование кейса показать ограничения проверки

Поле `alternative_explanations` существует именно потому, что кейс требует показывать ревьюеру ограничения метода, а не только основания.

4. Извлечение рубрики из условия

Роль: извлечь названия критериев и описания идеального результата. **Схема вывода:** `assignment_title, criteria[{name, max_score, requirements, indicators}]`.

Разделение труда здесь принципиально:

Что извлекается	Чем	Почему так
максимумы баллов, объём, кегль, шрифт, интервал, срок, штраф, окно досдачи	регулярками	в условии записаны однозначно; регулярка на этом не ошибается, а модель может
названия критериев, описания идеального результата, проверяемые признаки	моделью	связный текст, нужна семантика

При расхождении **приоритет у регулярок**: `max_score` берётся из распознанных цифр, а не из ответа модели. Несовпадение числа критериев и числа найденных максимумов попадает в `notes` рубрики и показывается методисту.

Правило промпта	Зачем
Извлекать только явно написанное, ничего не добавлять	модель дописывала критерии «по смыслу курса»
Названия и описания переносить дословно	пересказ ломал соответствие рубрики условию
Не менять порядок и не объединять критерии	порядок задаёт соответствие максимумам из регулярок

Результат — всегда черновик. `approved` остаётся `false`, и ревью по неутверждённой рубрике не запускается (`RubricNotApproved` → HTTP 400): ошибка в рубрике искажила бы все баллы разом.

Обработка ошибок модели

Требование кейса «описать обработку ошибок моделей».

Отказ	Поведение
Невалидный JSON	ремонтный ретрай: модели показывают её ответ и текст ошибки валидации (до <code>LLM_MAX_RETRIES</code> раз)
Провайдер не знает формат	деградация <code>json_schema</code> → <code>json_object</code> → свободный текст с извлечением JSON
Ответ обёрнут в <code>```json</code> или несёт преамбулу	вырезается перед разбором
Пустой <code>content</code> (рассуждающая модель)	ловится smoke-тестом; лечится <code>LLM_REASONING_EFFORT=none</code>
Разгон генерации	<code>max_tokens</code> как жёсткий предохранитель
Таймаут	шаг помечается отказавшим, ревью продолжается
Цитаты превысили потолки	обрезаются, ответ не отвергается
Балл выше максимума критерия	ограничивается кодом
Отказ по одному критерию	критерий помечается <code>failed</code> , остальные оцениваются
Отказ обязательного шага	пайплайн останавливается, результат возвращается с пометкой «проверить вручную»
Вызов наружу при офлайн-контуре	<code>OutboundBlocked</code> → HTTP 502 с подсказкой про <code>.env</code>

Ни один из перечисленных отказов не роняет процесс и не оставляет работу без следа: всё попадает в трассу, видимую ревьюеру.

Защита данных и обработка ошибок моделей

docs/security.md

Ограничения кейса: не передавать персональные данные студентов, внутренние материалы и код в незащищённые сервисы; описать меры защиты данных, обработку ошибок моделей и участие человека в итоговом решении.

1. Офлайн-контур

Приложение в рантайме не выходит в интернет. Единственный внешний адрес — `http://localhost:11434` (локальная Ollama). Это не декларация, а проверяемое свойство.

Как обеспечивается

Каждое обращение к модели проходит через страж в `app/privacy/audit.py`:

```
audit.guard(self.base_url, purpose="llm.chat") # до вызова
```

Хост вне `ALLOWED_HOSTS` → `OutboundBlocked`, вызов **не выполняется**, факт блокировки попадает в журнал. При `OFFLINE_ENFORCE=true` это жёсткий отказ, а не запись в лог.

Как проверяется

Способ	Где
Тест: попытка обращения к <code>api.openai.com</code> обязана бросить исключение	<code>tests/test_privacy.py::test_outbound_to_foreign_host_is_blocked</code>
Баннер в интерфейсе: внешних вызовов: 0 — число из фактически перехваченных вызовов	шапка всех вкладок
Вкладка «Приватность»: журнал последних вызовов с хостом, целью и длительностью	<code>GET /api/privacy</code>

Профиль `configs/providers/openai.env` приведён как доказательство провайдеро-независимости, и он **физически не заработает**, пока `api.openai.com` не добавлен в `ALLOWED_HOSTS` осознанно. Случайная утечка через смену модели невозможна.

Интернет нужен ровно один раз — в `scripts/setup.ps1` (установка пакетов, загрузка модели, сборка интерфейса, выгрузка примеров).

2. ПДн-щит

Почему это не декоративный модуль

Проверено на реальных примерах работ: в метаданных `.docx` лежат настоящие ФИО авторов. Работа, отправленная в модель «как есть», уносит персональные данные вместе с `docProps/core.xml`.

Что маскируется

Слой	Что находит
Регулярные выражения	email, телефон РФ, ник мессенджера, ссылка, ИНН, СНИЛС, паспорт, номер карты (с проверкой Луна), дата рождения
NER natasha	имена людей, которые формат не выдаёт — с проверкой разборщиком имён
Метаданные документа	author, lastModifiedBy — отдельным проходом

Ключевые решения

Маскирование выполняется ДО вызова модели, всегда. Это отдельный обязательный шаг пайплайна: если он не отработал, обращение к модели не происходит вовсе.

Номер карты проверяется алгоритмом Луна. Без этого под маску уходило бы любое 16-значное число, включая суммы в расчёте ROI — то есть содержательную часть работы. Проверено тестом: 1500000, 900000, 250000 остаются в тексте, а тестовый 4111 1111 1111 1111 маскируется.

Одинаковое значение получает один и тот же псевдоним, разные — разные. Иначе в тексте распадаются связи между упоминаниями одного человека. Обратное тоже обязательно: пока счётчик псевдонимов был сломан, два разных человека получали один и тот же ЛИЦО_1, и обратная подстановка возвращала студенту чужое имя. Закреплено тестом test_different_people_get_different_pseudonyms.

Найденное NER имя проверяется разборщиком имён. NewsNERTagger уверенно помечает PER не только людей: на реальной работе под маску уходило название продукта («Автотека») и заголовок таблицы («Верхнеуровневая CJM»). Это не перестраховка, а порча: работу оценивают в том числе по корректности описания продукта, а модель видела на его месте псевдоним человека. Поэтому фрагмент дополнительно разбирается NamesExtractor на имя, фамилию и отчество — не разобрался, значит не человек.

Плата названа прямо: PER -фрагмент, который разборщик не осилил (имя латиницей, редкая форма), маскироваться перестаёт. Риск ограничен — такие имена NewsNERTagger и без того не находит, форматные идентификаторы ловятся регулярками, а ФИО автора из метаданных заменяется по точному совпадению строки и через NER не проходит вовсе.

Замены обратимы. Карта псевдонимов локальна и применяется в обратную сторону при выдаче фидбека: студент видит своё имя, а не ЛИЦО_1.

Деградация явная. Нет natasha — слой имён отключается, работают регулярки, и в отчёт попадает предупреждение. Молчаливого ухудшения нет.

Как проверяется

Тест test_original_values_never_reach_the_model проверяет не «работает ли маскирование», а **отсутствие исходных значений в том, что фактически уходит в модель.** Это единственная формулировка, которая что-то доказывает.

Именно этот тест нашёл настоящую утечку: регулярное выражение для телефона не обрабатывало разделитель «пробел + скобка», и номер вида +7 (999) 123-45-67 уходил в модель незамаскированным.

3. Что и где хранится

Данные	Где	Покидает ли машину
Файлы работ	data/uploads/	нет

Данные	Где	Покидает ли машину
Результаты ревью	data/app.db (SQLite)	нет
Карта псевдонимов ПДн	в памяти процесса, на время ревью	нет
Журнал вызовов	в памяти процесса	нет
Примеры работ	data/examples/	нет

Каталог `data/` целиком исключён из репозитория (`.gitignore`): в итоговую выгрузку не попадают ни работы, ни база, ни `.env` .

В репозитории нет персональных данных. Демонстрационные пользователи синтетические (Анна Лебедева , Борис Мельник), в тестах и докстрингах — только вымышленные имена и контакты на `example.com` .

4. Обработка ошибок моделей

Подробная таблица — в `agent_instructions.md` . Короткий принцип: **ни один отказ модели не роняет процесс и не оставляет работу без следа.**

- невалидный JSON → ремонтный ретрай с текстом ошибки;
- провайдер не знает формат → деградация форматов;
- таймаут или разгон генерации → `max_tokens` и пометка отказа;
- отказ по одному критерию → остальные оцениваются, критерий помечен `failed` и виден ревьюеру;
- отказ обязательного шага → результат возвращается с пометкой «проверить вручную» и повышенным приоритетом.

Каждый отказ попадает в **трассу выполнения**, которую ревьюер видит в карточке работы. Он всегда знает, что именно система успела проверить.

5. Роль человека в итоговом решении

Кейс требует, чтобы AI не заменял ревьюера. В решении это обеспечено структурно — не обещанием в презентации, а кодом.

Решение	Кто принимает	Чем обеспечено
Итоговая оценка	ревьюер	студент видит балл только после <code>POST /confirm</code> ; до этого поле <code>final_score</code> пустое, а <code>published</code> равно <code>false</code>
Рубрика	методист	ревью не стартует при <code>rubric_approved = false</code> → HTTP 400. Правка рубрики сбрасывает утверждение
Сигнал генеративного ИИ	ревьюер	вклад идёт в индекс приоритета, не в балл ; вердикт <code>confirmed / rejected</code> фиксируется в БД
Непроверенная цитата	ревьюер	балл не снижается автоматически; критерий помечается и поднимает приоритет
Распределение работ	методист	автораспределение предлагает, ручной перенос доступен всегда
Текст обратной связи	ревьюер	поле редактируется до отправки

Что студент не получает

Фильтрация сделана **на сервере**, а не в интерфейсе: студенту не отправляются `priority_index`, `ai_score`, `review`, `blocks`, `summary`. Скрывать их разметкой было бы недостаточно — данные всё равно лежали бы в теле ответа. Проверено тестом сценария: у роли `student` этих полей в ответе нет.

Почему сигнал ИИ отделён от балла структурно

Кейс запрещает генеративной проверке автоматически влиять на оценку. Поэтому в `app/scoring/formula.py` считаются **два независимых числа**:

- **предварительный балл** — сумма по критериям минус штрафы;
- **индекс приоритета ручной проверки** — взвешенная сумма флагов риска.

Сигнал детектора входит **только во второе**. Тумблер детектора гасит его вклад в приоритет; в балл он не входит ни во включённом, ни в выключенном состоянии. Разделить их было нельзя «настройкой» — они посчитаны разными формулами из разных слагаемых.

6. Упрощения, принятые осознанно

Честный список — то, что в продакшене потребует доработки.

Упрощение	Почему допустимо в MVP	Что нужно для пилота
Вход по паролю есть, полноценной аутентификации нет. Пароль хранится хешем <code>pbkdf2_sha256</code> (200 000 итераций, индивидуальная соль), ответ на неверный логин и пароль одинаковый. Но сессия — идентификатор в заголовке: нет подписи, срока жизни и ограничения частоты попыток	демонстрация ролей и изоляции данных; контур защищён тем, что приложение не выходит в сеть	SSO организации, подписанные сессии, ограничение частоты попыток
Нет шифрования БД	локальная машина, данные не покидают контур	шифрование на уровне диска или БД
Журнал вызовов в памяти	демо — один процесс, история за сессию не нужна	вынести в таблицу с ротацией
Нет разграничения прав внутри роли	один поток, три ревьюера	права на уровне программы и курса
Псевдонимизация, а не удаление	нужна обратная подстановка в фидбеке	политика хранения карты замен

Об аутентификации сказано прямо на экране входа, чтобы упрощение не выглядело недоработкой.

Метрики качества и эффекта

docs/metrics.md

Пункт 6 постановки задачи: протестировать предварительное ревью не менее чем на трёх вариантах выполнения задания и представить результаты. Плюс критерий успеха «Эффективность»: сокращение числа ручных действий, времени обработки, количества просроченных проверок.

1. Результаты бенчмарка

Прогон: `python -m app.bench.run_bench --runs 3`. Данные: три предоставленных решения ДЗ «Карта рисков продукта». Отчёт: `data/bench_report.json`.

Работа	Средний балл	Разброс между прогонами	Формальных нарушений	Цитат подтверждено	Время
слабое	6,5 / 10	0	1	40 / 46 (87 %)	40 с
среднее	8,3 / 10	0	1	52 / 64 (81 %)	49 с
хорошее	8,0 / 10	0	0	63 / 66 (95 %)	47 с

Сводные показатели

Метрика	Значение
Правильность ранжирования	67 % — две пары из трёх упорядочены верно
Стабильность	разброс 0 — и итоговый балл, и балл по каждому критерию совпадают на всех трёх прогонах
Подтверждаемость цитат	88 % по всем работам
Среднее время на работу	45 с
Отказов шагов пайплайна	0
Неоценённых критериев	0

Разбор по парам работ — где система ошибается

Агрегат «67 %» скрывает главное, поэтому бенчмарк печатает разбор по парам:

Пара	Верно прогонов	Средний разрыв	Вывод
слабое < среднее	3 / 3	+1,8	разделены уверенно
слабое < хорошее	3 / 3	+1,5	разделены уверенно
среднее < хорошее	0 / 3	-0,3	НЕ РАЗДЕЛЕНЫ

Слабая работа отделяется от обеих сильных устойчиво и с большим отрывом. Среднюю и хорошую система не различает: разрыв 0,3 балла на десятибалльной шкале — это не обратный порядок, а отсутствие разрешающей способности.

Полный разбор по критериям — ниже; там видно, что весь разрыв даёт один критерий из пяти. Значения устойчивы: на трёх прогонах каждый критерий даёт одно и то же число. Варьируется только длина списка цитат у средней работы (20 или 22 доказательства) — балл при этом не меняется.

Почему среднее и хорошее не разделились

Причина локализована до одного критерия:

Критерий	Максимум	Слабое	Среднее	Хорошее
Корректность описания продукта	1	0,5	1,0	1,0
Глубина и системность анализа рисков	4	3,5	3,5	3,5
Обоснованная приоритизация	2	1,5	1,5	1,5
Реалистичный расчёт ROI	2	0,5	1,5	1,5
Качество оформления	1	0,5	0,8	0,5
Итого	10	6,5	8,3	8,0

По четырём критериям из пяти средняя и хорошая работы получают **ровно одинаковый балл**. Инверсию даёт единственный критерий — **«Качество оформления»**, и по существу дела модель здесь не ошибается: критерий требует «использованы заголовки, списки, таблицы, схемы», а у хорошей работы **0 таблиц и 0 заголовочных стилей** — её структура выражена нумерацией внутри обычных абзацев. У средней работы 4 таблицы. Формально средняя оформлена богаче.

Это ограничение метода, а не дефект реализации: ярлык «хорошее» в репозитории примеров отражает качество работы в целом, а рубрика на этом критерии измеряет наличие конкретных элементов оформления.

Почему главная метрика — ранжирование, а не абсолютный балл

Точных оценок преподавателя по этим работам у нас нет: репозиторий кейса задаёт только относительные уровни (слабое / среднее / хорошее). Поэтому измеряется **порядок**, а не попадание в конкретное число. Ранжирование — именно то, что требуется от помощника ревьюера: правильно расставить работы и показать, где искать проблемы.

Оценки по критериям (прогон 1)

Критерий	Максимум	Слабое	Среднее	Хорошее
Корректность описания продукта и ценностного предложения	1	0,5	0,5	1,0
Глубина и системность анализа рисков	4	3,5	3,5	3,5
Обоснованная приоритизация трёх критичных рисков	2	1,5	1,5	1,5
Реалистичный расчёт ROI митигации	2	0,5	1,5	1,5
Качество оформления	1	0,5	0,8	0,5
Итого	10	6,5	7,8	8,0

Разделение идёт по двум критериям: описание продукта и расчёт ROI. У слабой работы ROI фактически не рассчитан, и это главный вклад в разрыв.

Критерий «приоритизация» у всех трёх работ получает 1,5 из 2 по одной и той же причине: ни в одной нет **визуальной матрицы «Вероятность x Влияние»** с цветовой кодировкой, которую требует условие. Матрица нарисована картинкой либо отсутствует, а изображения текстовая модель не анализирует — это честно помечено в отчёте как «изображение не оценено».

2. Формальный слой различает работы

Слой А работает без модели и даёт полностью воспроизводимый результат. Три работы получают **три разных** набора нарушений.

Проверка	Слабое	Среднее	Хорошее
Объём	не определено (нет <code>app.xml</code> , оценка ~1 стр.)	✓ 3 стр.	✓ 3 стр.
Шрифт текста	✓ Arial	✓ Arial	✓ Arial + Cambria Math ¹
Шрифт в таблицах	✗ Arial Unicode MS	✓ Arial	— таблиц нет
Кегль текста	✓ 11 пт	✗ встречается 10 пт	✓ 11 пт
Кегль в таблицах	✓ 10 пт	✓ 10 пт	— таблиц нет
Межстрочный интервал	✓ 1,15	✓ 1,15	не определено ²
История изменений	не определено ³	✓	✓
Декларация об использовании ИИ	не найдена	не найдена	не найдена

¹ Cambria Math приходит из редактора формул — не считается нарушением. ² Интервал не задан ни в параграфах, ни в `styles.xml` , ни в `docDefaults` . Честный ответ — «не определено», а не «нарушение». ³ Метаданных нет вовсе; по локальной копии отсутствие истории не доказывается.

Что показал этот слой при разработке

Первоначально зафиксированный факт «слабое решение: Arial Unicode MS , весь текст 10 пт» оказался артефактом наивного grep по `document.xml` : из 155 вхождений `sz=20 82` лежат в `w:pPr/w:rPr` — это свойства метки параграфа, на текст они не влияют. С разрешением наследования стилей картина другая: у слабого решения основной текст соответствует требованиям, а нарушение — только в шрифте таблиц. Слой всё равно различает работы, но по другим признакам, чем предполагалось.

3. Как метрики качества менялись

Все правки — исправления наблюдавшихся дефектов, а не подкрутка под известные работы. Важная оговорка: все строки до последней измерялись на устаревшем кэше рубрики и потому не отражают поведение приложения — см. поправку ⁵.

Что изменено	Слабое	Среднее	Хорошее	Цитат	Время худшей работы
Исходно	7,0	7,5	8,0	—	292 с
Убраны служебные поля из схемы цитат ¹	7,0	8,0	8,0	77 %	86 с
Добавлена дисциплина оценки ²	5,2	7,5	8,0	77 %	86 с
Блок формата вынесен в начало промпта ³	6,7	8,3	8,5	79 %	50 с
Дисциплина повторена в последней строке ⁴	6,5	7,8	8,0	89 %	49 с
Кэш рубрики инвалидирован ⁵	6,5	8,1	8,0	88 %	50 с
Исправлена точность ПДн-щита⁶	6,5	8,3	8,0	88 %	49 с

¹ Модель заполняла `status` и `similarity` сама, ответ упирался в потолок токенов и обрывался на середине JSON, вызывая до 9 ремонтных попыток на критерий. Разделение типов `EvidenceIn` / `Evidence` устранило это структурно — и заодно отобрало у модели возможность утверждать проверку, которую она выполнить не может.

² Рубрика описывает только идеальный результат, и без правила частичного начисления модель начисляла балл «снизу вверх», завышая слабые работы. Правило сформулировано принципиально («максимум только при выполнении всех признаков», «заявленное но не раскрытое — не выполнено»), а не подогнано под эти три работы.

³ Если промпт заканчивается схемой, модель продолжает её и возвращает саму схему вместо данных: `{"properties": {...}, "required": [...], "title": "..."} — формально JSON, но не значения. Наблюдалось на самой короткой из работ, где из-за этого полностью терялся сигнал судьи по гениИИ (вес 0,35 в ансамбле).`

⁴ Побочный эффект правки ³: задача сместилась из конца промпта в середину, и дисциплина оценки ослабла — слабая работа получила 6,7. Дисциплина повторена в последней строке промпта критерия, потому что последняя строка влияет сильнее всего.

⁵ **Самая важная поправка, и она ухудшила заявленный результат.** Бенчмарк кэшировал извлечённую рубрику в файл с фиксированным именем. После правок промпта извлечения кэш перестал соответствовать тому, что производит приложение: у критерия «Качество оформления» в кэше было 5 проверяемых признаков, а в свежей рубрике 9. В итоге **бенчмарк измерял конфигурацию, которой в продукте не существует**, и показывал 100 % ранжирования, тогда как приложение на тех же работах давало 67 %.

Расхождение обнаружилось при сквозном прогоне через интерфейс: приложение выдало 8,1 у средней работы против 7,8 в бенчмарке. Ключ кэша теперь включает отпечаток промптов извлечения и файла условия, поэтому такое расхождение больше невозможно. Все числа в документации приведены к измерению на актуальной конфигурации.

Честный итог

Наилучшее формальное разделение (разрыв 2,8 балла между слабой и хорошей) давал вариант со схемой в конце промпта — но он терял сигнал судьи по гениИИ, давал 77 % подтверждаемости цитат и работал 86 секунд на работу. Текущая конфигурация надёжно отделяет слабую работу (разрыв 1,5–1,8 балла), не различает среднюю и хорошую, даёт 88 % цитат и 45 секунд.

Дальнейший подбор формулировок промпта на трёх известных работах был бы подгонкой под тестовый набор, а не улучшением качества, поэтому мы остановились здесь. Для помощника ревьюера устойчивое выделение слабых работ — практически более ценное свойство, чем различение двух сильных: именно слабые требуют внимания человека в первую очередь.

⁶ NER относил к людям название продукта («Автотека», «Автотеки») и заголовок таблицы «Верхнеуровневая CJM», и в модель на месте продукта уходил `ЛИЦО_1`. Второй фильтр — разбор фрагмента `NamesExtractor` — это устранил.

Замер получился редким по чистоте: изменился **ровно один критерий ровно у одной работы** — «Корректность описания продукта» у средней, 0,8 → 1,0. Остальные четыре критерия средней работы, а также все критерии слабой и хорошей, не сдвинулись ни на сотую. Хорошая работа здесь контроль: у неё ПДн-щит находил только ссылки, названия продукта под маску не попадали — и её баллы не изменились.

Это и есть причинно-следственная проверка гипотезы: если модель не видит названия продукта, страдает именно тот критерий, который спрашивает про описание продукта. Побочный эффект честный — разрыв в неразделённой паре вырос с −0,1 до −0,3, потому что выиграла как раз средняя работа.

Дефект нашёлся не тестом, а тем, что подсветку персональных данных вывели в интерфейс: пока маскирование было невидимым, ошибка выглядела как норма.

3а. Второй контрольный набор — курс по QA

Основной сценарий даёт три работы одного задания. Чтобы понять, где выводы верны, а где это особенность одного условия, тот же пайплайн прогнан на техническом QA: другой курс, другой формат решений (`.xlsx`), другая рубрика (5 критериев, максимум 20).

Работа	Балл	Цитат подтверждено	Время
слабое	12,5 / 20	84 %	60 с
среднее	15,5 / 20	96 %	115 с
хорошее	17,3 / 20	71 %	78 с

Ранжирование 100 %: все три пары упорядочены верно, включая пару «среднее — хорошее», которая на основном сценарии не разделяется. Это показывает, что неразделимость там — свойство конкретного условия (критерий оформления требует таблиц, которых у хорошей работы нет), а не общий предел подхода.

Что этому предшествовало

Первый прогон на том же наборе дал **13,5 / 15,5 / 15,0** — порядок нарушен, средняя выше хорошей. Причина оказалась не в модели: читатель `.xlsx` делал из листа один блок `§N`, и на хорошей работе это был блок на 19 тысяч символов. Проверка цитат ищет цитату внутри указанного блока — в блоке такого размера она находит почти любую строку.

После резки листа на блоки по строкам (с повтором шапки таблицы):

	до	после
слабое	13,5	12,5
среднее	15,5	15,5
хорошее	15,0	17,3
порядок	нарушен	верный
время на работу	277 с	60–115 с

Вывод, который мы из этого сделали: размер блока `§N` — не деталь парсера, а параметр качества оценки.

3б. Третий контрольный набор — системный дизайн

Технический курс, условие и решения в Markdown, рубрика из девяти критериев по 0,5–1 баллу, максимум 6.

Работа	Балл	Цитат подтверждено
слабое	5,7 / 6	79 %
среднее	4,0 / 6	100 %
хорошее	5,75 / 6	76 %

Ранжирование 67 % — две пары из трёх. Хорошая работа наверху, средняя внизу; слабую от хорошей система не отличает (5,7 против 5,75).

Как эта цифра выросла с 33 %

Первый прогон дал 4,5 / 3,0 / 5,5, второй — 5,7 / 3,7 / 4,75, то есть ранжирование 33 %. Разбор показал два дефекта, и оба чинились в коде, а не подбором весов.

Дефект первый: требования чужого курса. `FormalRequirements` держал пороги ДЗ по продуктовому фроду как значения по умолчанию, и работа в Markdown теряла балл за «формат MD не входит в разрешённые (DOCX, PDF)» — требование, которого в её условии нет. Условие системного дизайна прямо разрешает Markdown.

Дефект второй: разорванные диаграммы. Решения содержат схемы mermaid, внутри которых есть пустые строки. Читатель `.md` резал текст по пустой строке и разбивал диаграмму на четыре блока: модель видела обрывки `subgraph ...` вместо схемы. Работа с тремя диаграммами получала **ноль** за критерий «Построена диаграмма C4».

После починки блок в тройных кавычках остаётся одним блоком `SN`, тип диаграммы определяет код (`C4Context` → «C4 контекстная»), и факт уходит в промпт наравне со шрифтами и таблицами. Результат по критерию диаграмм у хорошей работы: **0 → 1,0 балла**, общий балл 4,75 → 5,75.

Что осталось и почему

Слабое решение (5,7) и хорошее (5,75) получают **одинаковый балл по восьми критериям из девяти**. Причина видна в самой рубрике: её критерии спрашивают о **наличии** — «выполнена декомпозиция», «определены интерфейсы», «построена диаграмма», — а не о качестве архитектурного решения. Слабое решение объёмом 23 тысячи символов, с четырьмя диаграммами и описанными контрактами, по такому чек-листу неотличимо от хорошего.

Это ограничение рубрики, а не только нашей системы: ревьюер-человек по этим девяти пунктам поставил бы похожие баллы и различил бы работы по существу архитектуры. Подгонять систему под нужный ответ мы не стали.

Что это значит вместе с остальными курсами

Курс	Формат решений	Критериев	Ранжирование
Технический QA	<code>.xlsx</code>	5	100 %
Продуктовый фрод	<code>.docx</code>	5	67 %
Системный дизайн	<code>.md</code>	9	67 %

Вывод честнее, чем «система различает работы»: **качество различения зависит от рубрики**. Там, где критерии спрашивают о качестве («реалистичный расчёт», «обоснованная приоритизация»), система различает работы; там, где они спрашивают о наличии («построена диаграмма», «определены интерфейсы»), объёмная слабая работа неотличима от хорошей. Для помощника ревьюера это приемлемо — итог всё равно ставит человек, — но обещать равномерное качество на всех курсах мы не можем и не будем.

4. Воспроизводимость

Условие	Как обеспечено
<code>temperature = 0</code>	<code>.env</code> и Modelfile
фиксированный <code>seed = 42</code>	то же
штрафы сэмплирования погашены	Modelfile: <code>presence_penalty 0, repeat_penalty 1, top_k 0</code>
версия модели зафиксирована	производная модель из Modelfile
фактические параметры проверяются	smoke-тест читает <code>/api/show</code> , а не <code>.env</code>

Результат: разброс баллов между тремя прогонами — ноль. Проверено и в CLI, и через веб-приложение: одна и та же работа получает 6,5 / 8,3 / 8,0 в обоих путях. Совпадение путей — отдельная проверка:

расхождение между ними однажды выявило устаревший кэш рубрики (см. поправку ⁵ выше).

Базовая модель `qwen3.5` приносит из своего Modelfile `presence_penalty 1.5`. При `temperature=0` выбор жадный, но штрафы всё равно правят logits до `argmax`, а структурный JSON состоит из повторяющихся токенов — кавычек, скобок, имён полей. Без гашения штрафов воспроизводимость и валидность страдали бы обе.

5. Метрики эффекта

Формулировки взяты дословно из критериев успеха кейса.

Число ручных действий

Базовая линия выведена в `as-is-to-be.md` из семи шагов процесса AS-IS: **11 действий на одну работу**.

Расчётная оценка (по шагам процесса):

	На работу
Было (AS-IS)	11
Стало (TO-BE)	3
Сокращение	≈ 73 %

Фактическое измерение на сквозном прогоне демонстрационного потока. Каждое действие пользователя пишется в `ActionLog`, и вкладка «Аналитика» считает отношение по журналу, а не по таблице выше:

Показатель	Продуктовый фронт	Технический QA
Работ прошло через процесс	3	3
Базовая линия (3 × 11)	33	33
Фактических действий пользователей	10	14
Сокращение	70 %	58 %

Минимальный набор — семь действий: загрузка условия, утверждение рубрики, распределение, массовый запуск ревью и три подтверждения ревьюером. Числа выше семи — это реальная работа методиста: повторные прогоны ревью, правка весов, заведение участников. На курсе QA заводились с нуля методист, ревьюер и трое студентов — эти действия разовые, но честно попали в знаменатель одного потока из трёх работ. На потоке из тридцати работ они амортизируются, и сокращение растёт.

Метрика считается по одному заданию. Числитель и знаменатель обязаны описывать одни и те же работы; когда журнал брался целиком, а базовая линия — по выбранному заданию, появление второго курса дало «было 33, стало 45, сокращение –36 %». Закреплено тестами `test_analytics_scope.py`.

Что в знаменатель НЕ входит. Базовая линия считается только по работам, реально прошедшим процесс. В демонстрационных данных есть три строки без файла — это нагрузка прошлого потока, которая занимает ёмкость ревьюера, но через проверку в этом потоке не проходила. Включение их в знаменатель дало бы 82 % вместо 70 %, то есть зависило бы эффект. Метрику должно быть можно оспорить, а не только показать.

Время обработки

Показатель	Значение
Предварительное ревью одной работы (машина)	45 с , в фоне
Время ревьюера на работу: было	15–25 мин (из оценки кейсодателя 5–10 ч/нед)

Показатель	Значение
Время ревьюера на работу: стало	5–8 мин — проверить готовое и подтвердить
Время до получения фидбека студентом	сразу после подтверждения (SSE), вместо ожидания переноса в таблицу

Машинное время не конкурирует с человеческим: очередь обрабатывается в фоне, и к моменту открытия работы ревью готово.

Количество просроченных проверок

Считается автоматически: `review_status()` сравнивает срок проверки с моментом подтверждения. Состояния — `overdue`, `overdue_closed`, `due_soon`, `done`. Метрика выводится на вкладке «Аналитика» и уходит в выгрузку.

До внедрения этот показатель вообще не измерялся — просрочки замечались вручную при просмотре таблицы.

Качество автоматики

Метрика	Как считается
Согласие ревьюера с AI	доля подтверждений без правки балла (<code>score_edited = false</code>)
Доля критериев без подтверждённых цитат	из результата ревью
Средний индекс приоритета	из результата ревью

Согласие ревьюера — главная метрика доверия к системе на пилоте. Механизм подсчёта готов и работает: на сквозном прогоне он показал 100 % (3 из 3).

Это число не является результатом качества. В прогоне подтверждение выполнялось программно, без правки баллов, — то есть измерен сам механизм, а не согласие живого ревьюера. Осмысленное значение метрики даст только пилот с реальными проверяющими на десятках работ.

6. Проверенные сценарии сроков и штрафов

Правило из условия проверено сквозным прогоном через приложение: сроки сдвинуты так, чтобы три работы оказались в трёх разных состояниях.

Работа	Сдана	Состояние	Штраф	Балл
слабое	до мягкого срока	<code>on_time</code>	0	5,2 → 5,2
среднее	через 35 мин после мягкого срока	<code>late_penalty</code>	–1	7,5 → 6,5
хорошее	после жёсткого срока	<code>late_zero</code>	обнуление	8,0 → 0,0

Пересчёт выполнен планировщиком **в течение 3 секунд** после изменения сроков, уведомления доставлены студенту и ревьюеру по SSE.

Отдельно проверено: работа, сданная **до** дедлайна, не становится просроченной, если методист позже сдвинул срок вперёд. Состояние сданной работы определяется моментом сдачи, а не текущим временем.

7. Автоматические тесты

`python -m pytest -q` — **132 теста, 7 с**. Восемь из них (`test_smoke_llm.py`) требуют запущенного провайдера и пропускаются без него.

Набор	Тестов	Что проверяет
<code>test_smoke_lm.py</code>	8	фактический <code>num_ctx</code> из <code>/api/show</code> ; отключённость рассуждений; детерминизм; structured output; ремонтный ретрай; офлайн-страж
<code>test_formal.py</code>	10	три работы дают три разных набора нарушений; неопределённость не становится нарушением; отсутствие каскадных отказов
<code>test_evidence.py</code>	9	выдуманная цитата отбраковывается ; косметические расхождения переживаются; модель не может сама объявить проверку пройденной
<code>test_privacy.py</code>	22	исходные ПДн не попадают в запрос к модели; суммы не путаются с картами; название продукта не маскируется как человек; разные люди получают разные псевдонимы; деградация без NER; офлайн-контур
<code>test_deadlines.py</code>	9	правило штрафа; четыре состояния; демо-масштаб в секундах; naïve-даты из SQLite
<code>test_workload.py</code>	15	перекося нагрузки выравнивается; компетенции; повторное ревью; схожесть работ
<code>test_events.py</code>	7	адресация событий по роли; медленный подписчик не блокирует остальных
<code>test_scoring.py</code>	9	сигнал гениИ не влияет на балл ни при каком положении тумблера ; непроверенная цитата поднимает приоритет, но не снижает балл; штраф применяется однократно
<code>test_llm_parsing.py</code>	9	эхо схемы разворачивается; markdown-ограждения и рассуждения вырезаются; служебные поля не попадают в схему модели
<code>test_pipeline_guards.py</code>	9	скан без текстового слоя не получает балл ; непроверяемая работа уходит в начало очереди; работа сверх бюджета контекста обрезается с предупреждением
<code>test_ingest.xlsx.py</code>	8	лист не превращается в один блок; шапка таблицы повторяется в каждом блоке; блок сообщает, какие это строки книги
<code>test_scheduler.py</code>	3	напоминание отправляется один раз, а не каждую секунду ; сводка методисту молчит, пока поток не изменился

Тесты, нашедшие настоящие дефекты

Часть набора написана после того, как проверка нашла ошибку, — и закрепляет именно её. Ошибки, которые иначе проявились бы на защите:

- `test_original_values_never_reach_the_model` — регулярное выражение для телефона не обрабатывало разделитель «пробел + скобка», и номер вида `+7 (999) 123-45-67` уходил в модель **незамаскированным**.
- `test_subscriber_is_hashable` — `Subscriber` был обычным `@dataclass`, поэтому Python выставлял `__hash__ = None`, и хранение в `set` падало. Внешне это выглядело как «интерфейс не обновляется».
- `test_naive_due_date_does_not_crash` — SQLite отдаёт даты без часового пояса, и распределение отвечало 500 после первого перезапуска приложения.
- `test_different_people_get_different_pseudonyms` — счётчик псевдонимов перебирал значения карты замен вместо ключей и всегда оставался нулём: разные люди получали один и тот же `ЛИЦО_1`, а обратная подстановка возвращала студенту чужое имя.
- `test_review_reminder_is_sent_once_not_every_tick` — коммит в тике планировщика стоял под условием «изменилось состояние работы», а напоминания создаются именно в тиках без изменений: уведомление уходило по SSE, но откатывалось, и отправлялось заново каждую секунду.
- `test_scan_without_text_layer_is_not_scored` — работа, сданная сканом, уходила в модель пустым документом, и система выставляла балл документу, которого не видела.

7. `test_long_sheet_is_split_into_several_blocks` — лист `.xlsx` становился одним блоком на 19 тысяч символов, и проверка цитат внутри него теряла смысл.

8. Честные ограничения бенчмарка

- **Три работы одного задания** — показатель направления, а не статистика. Для оценки качества как метрики нужна разметка ревьюерами на десятках работ.
- **Эталонных баллов нет** — сравнивается порядок, а не абсолютные значения.
- **Схемы и изображения не анализируются.** У всех трёх работ матрица рисков либо картинка, либо ссылка — и это прямо влияет на оценку критерия приоритизации.
- **Подтверждаемость цитат 52 % у среднего решения** объясняется структурой: таблицы там склеены в крупные блоки, и модель цитирует их с переформатированием. Механизм работает правильно — он честно сообщает, что цитата не подтверждена, и поднимает приоритет ручной проверки.
- Метрики эффекта считаются на демонстрационном потоке. Базовая линия выведена из описания процесса в кейсе, а не измерена секундомером у ревьюеров.

Стоимость эксплуатации

docs/cost.md

Ограничение кейса: при выборе моделей и сервисов учитывать стоимость эксплуатации и обосновать баланс между качеством, скоростью и затратами.

1. Замеренные показатели

Измерено на реальных данных: три работы ДЗ «Карта рисков продукта», три прогона. Оборудование — RTX 5070 Ti 16 ГБ, 61 ГБ RAM.

Показатель	Значение
Модель	qwen3.5:9b Q4_K_M через Ollama, контекст 32 768
Занято VRAM	9,9 ГБ из 15,9 (100 % слоёв на GPU)
Вызовов модели на одну работу	7 (5 критериев + судья генИИ + фидбек)
Время на один критерий	3–10 с
Время полного ревью одной работы	39–49 с (среднее ≈ 45 с)
Токенов на входе	≈ 2 800 на критерий
Токенов на выходе	200–1 050 на критерий
Скорость генерации	≈ 86 токенов/с

Разброс времени объясняется размером работы: 3,4 тыс. символов против 9,1 тыс. Все работы целиком укладываются в контекст, map-reduce не требуется.

2. Локальная модель против облачных API

Расчёт на **1 000 работ** — примерный масштаб потока одного курса за сезон.

Расход токенов на 1 000 работ

При 7 вызовах на работу и измеренных объёмах:

	На работу	На 1 000 работ
Входные токены	≈ 20 000	20 млн
Выходные токены	≈ 3 500	3,5 млн

Стоимость

Вариант	Разовые затраты	На 1 000 работ	Данные покидают контур
Локально, Ollama + qwen3.5:9b (текущий)	GPU уже есть	0 ₽	нет
Локально, аренда GPU-сервера	—	≈ 2 000 ₽ (20 ч × 100 ₽/ч)	нет
Собственный vLLM в контуре	развёртывание	амортизация железа	нет
Облачный API уровня GPT-4o mini	—	≈ 5 500 ₽	да
Облачный API уровня GPT-4o	—	≈ 90 000 ₽	да

Оценки облачных вариантов даны по порядку величины и приведены для сравнения: точные тарифы меняются, а вывод от их колебаний не зависит.

Почему выбран локальный вариант

- 1. **Требование кейса.** Передача персональных данных студентов в незащищённые сервисы запрещена. В работах реальные ФИО — проверено. Локальная модель снимает вопрос целиком, а не смягчает его.
- 2. **Нулевая переменная стоимость.** Рост потока не увеличивает счёт. Это принципиально для процесса, который по условию задачи должен масштабироваться.
- 3. **Предсказуемость.** Нет тарифов, лимитов, недоступности провайдера и изменения поведения модели без предупреждения.
- 4. **Воспроизводимость.** Версия модели зафиксирована, `temperature=0` , `seed` задан. Облачные модели обновляются, и прошлое ревью перестаёт воспроизводиться.

3. Баланс качества, скорости и затрат

Почему 9B, а не крупнее и не мельче

Вариант	Качество	Скорость	Помещается в 16 ГБ	Решение
<code>qwen3.5:4b</code>	формулировки заметно грубее, разделение работ не проверялось систематически	≈ вдвое быстрее	да, с запасом	оставлен как <code>LLM_MODEL_FAST</code> для черновых прогонов
<code>qwen3.5:9b Q4_K_M</code>	слабая работа отделяется устойчиво, цитат 88 %	45 с на работу	9,9 ГБ с контекстом 32k	выбран
30B+	предположительно выше	не помещается целиком → выгрузка слоёв в RAM, падение в разы	нет	отклонён

Решающий довод: 9B — крупнейшая модель, которая помещается на GPU **целиком вместе с контекстом 32k**. При выгрузке части слоёв в RAM время ревью выросло бы в разы, и очередь перестала бы обрабатываться за разумное время.

Где скорость получена бесплатно

Две находки дали основной выигрыш, не затронув качество:

Что	До	После
Отключение рассуждений. <code>qwen3.5</code> — рассуждающая модель; при включённых рассуждениях вывод уходит в поле <code>thinking</code> , <code>content</code> остаётся пустым, генерация не останавливается	тривиальный запрос уходил в таймаут 180 с; smoke-набор 398 с	ответ 0,2–0,7 с; smoke-набор 2,7 с
Убраны служебные поля из схемы цитат. Модель заполняла <code>status</code> и <code>similarity</code> , ответ упирался в потолок токенов и обрывался на середине JSON, вызывая полный цикл ремонтных ретраев	среднее решение — 292 с, до 9 попыток на критерий	86 с , все вызовы с первой попытки

Итог: время ревью одной работы сократилось с 292 с до 70 с в худшем случае — **вчетверо**, без потери качества. Экономия достигнута исправлением конфигурации и схемы, а не упрощением проверки.

Стоимость в человеко-часах

Прямое сравнение с базовой линией процесса из `as-is-to-be.md` .

	AS-IS	TO-BE
Ручных действий на работу	11	3
Время ревьюера на работу	15–25 мин (оценка кейсодателя: 5–10 ч/нед)	5–8 мин (проверить готовое ревью и подтвердить)
Время машины на работу	—	70 с (параллельно, ревьюер не ждёт)
Перенос результатов в таблицу	3 действия руками	0 — выгрузка по кнопке

Машинное время не конкурирует с человеческим: очередь обрабатывается в фоне, и к моменту, когда ревьюер открывает работу, ревью готово.

4. Эксплуатационные требования

Ресурс	Минимум	Комментарий
GPU	12 ГБ VRAM	для 9B с контекстом 32k; на 8 ГБ следует перейти на <code>4b</code>
RAM	16 ГБ	сама модель на GPU
Диск	15 ГБ	модели $\approx$ 10 ГБ, база и работы — мегабайты
Процессов	1	<code>uvicorn</code> ; Node на демонстрации не нужен
Внешние сервисы	нет	ни БД, ни брокера, ни кэша

Отсутствие инфраструктуры — тоже часть стоимости: нет Docker, Redis, Celery и отдельного сервера БД, значит нет их настройки, мониторинга и отказов.

5. Путь на прод

Шаг	Что меняется	Стоимость
Пилот на одном курсе	ничего, текущая конфигурация	0
Несколько курсов, один сервер	<code>REVIEW_WORKERS</code> и ёмкости ревьюеров в интерфейсе	0
Нагрузка выше одной GPU	<code>LLM_BASE_URL</code> → внутренний vLLM; батчинг запросов	сервер с GPU
Требуется качество выше	модель крупнее на том же vLLM	правка <code>.env</code> , кода нет
SQLite стал узким местом	<code>DB_URL</code> → PostgreSQL (SQLAlchemy уже async)	сервер БД

Ни один шаг не требует переработки архитектуры: смена провайдера и базы — это правка `.env`.

Масштабирование

docs/scalability.md

Критерий кейса: решение адаптируется к другим курсам, платформам, форматам заданий и количеству пользователей **без полной переработки архитектуры**.

1. Добавление нового ДЗ или курса — без единой строки кода

Это не оценка, а проверяемое утверждение: путь целиком проходит через интерфейс методиста.

1. Создать задание	→ «+ Новое задание» (название, направление, % ДЗ, канал)
2. Загрузить условие (PDF / DOCX / MD)	→ «Загрузить условие»
3. Рубрика извлечена автоматически	→ критерии, максимумы, формальные требования, сроки
4. Проверить и при необходимости поправить	→ «Править вручную» (JSON-редактор)
5. Утвердить	→ «Утвердить рубрику»
6. Добавить участников	→ «Участники»: ревьюеры с ёмкостью и компетенциями, студенты
7. Загрузить работы	→ «Загрузка работ»
8. Распределить и запустить ревью	→ готово

Все восемь шагов — кнопки в интерфейсе методиста. Ни один не требует правки кода, конфигов или обращения к API вручную.

Что при этом переносится автоматически из условия нового задания:

Извлекается	Чем	Куда попадает
критерии, максимумы, описания идеала	модель + регулярки	рубрика
объём в страницах	регулярка	формальная проверка «Объём»
шрифт и кегль текста и таблиц	регулярка	проверки шрифта и кегля
межстрочный интервал	регулярка	проверка интервала
срок сдачи, окно досдачи, штраф, срок проверки	регулярка	модель сроков и автопересчёт штрафа

Формальные требования не зашиты в код: `FormalRequirements` собирается из рубрики методом `Rubric.to_formal_requirements()`. Курс с другими требованиями (Times New Roman 14 пт, 5 страниц) получит те же проверки с другими порогами — это покрыто тестом `test_requirements_are_configurable`.

2. Другой формат решений

В проекте два курса с принципиально разными форматами:

Курс	Условие	Решения	Читатель
Продуктовый фрод (основной)	<code>.pdf</code>	<code>.docx</code> — связный текст	<code>docx_reader</code>
Продуктовая бизнес-модель	<code>.pdf</code>	<code>.xlsx</code> — таблицы с формулами	<code>xlsx_reader</code>

Оба сводятся к одному типу `Document` с блоками `§N`. Всё, что выше по течению — ревью, проверка цитат, подсветка, детектор, выгрузка — про формат не знает.

Проверено на файлах второго курса: три решения читаются, распознаётся 157 / 332 / 588 формул. Наличие формул — отдельный факт `has_formulas`, прямо относящийся к критерию «реалистичный расчёт»: он отличает живой расчёт от вписанных руками чисел.

Добавление формата = функция, возвращающая `Document`, плюс строка в `READERS`:

```
READERS: dict[str, Callable[[str | Path], Document]] = {
    ".docx": read_docx,
    ".pdf": read_pdf,
    ".xlsx": read_xlsx,
    ".ipynb": read_notebook,    # ← Data Science
}
```

Правок в других модулях не требуется.

3. Другой канал сдачи

MVP использует Stepik (условие — печать страницы Stepik в PDF, что видно по `/Producer (Skia/PDF)`). Канал влияет только на то, **как файл попадает в систему**, и изолирован в одном месте — загрузке.

Канал	Что добавить	Что не меняется
Stepik (текущий)	загрузка файла	—
Google Docs	экспорт по ссылке → тот же <code>read_any</code>	ревью, рубрика, интерфейс
GitHub Pull Request	обход файлов репозитория + diff версий	ревью, рубрика, интерфейс
Загрузка студентом	эндпоинт уже есть (<code>POST /submissions</code> доступен роли <code>student</code>)	всё

Повторное ревью уже учтено в модели данных: `WorkItem.is_repeat` и `previous_reviewer_id` заставляют распределение сохранить прежнего ревьюера — контекст уже у него в голове.

4. Другое направление и техническая проверка

Реестр направлений задаётся данными, а не кодом. Компетенции ревьюеров — список строк в профиле, распределение сверяет тег работы с этим списком.

Определение направления по содержимому — таблица ключевых слов `_TRACK_MARKERS` в `app/api/routes.py`. Здесь намеренно нет модели: правило простое и объяснимое, а ошибка дорога — она молча увела бы работу в чужой поток. Поэтому тег направления при загрузке **обязателен**, а определение системы служит только предупреждением: решает человек.

Технические курсы (QA, системный дизайн, Go) отличаются форматом условия (`.md`) и решений (`.xlsx`, репозитории). Читатель `.md` уже есть.

5. Рост числа пользователей и работ

Масштаб	Что делать	Правки кода
до ~100 работ на поток	ничего, текущая конфигурация	нет

Масштаб	Что делать	Правки кода
несколько курсов одновременно	ёмкости ревьюеров в интерфейсе; распределение учтёт	нет
очередь не успевает	<code>REVIEW_WORKERS</code> в <code>.env</code> ; на одной GPU смысла нет — Ollama сериализует запросы	нет
нагрузка выше одной GPU	<code>LLM_BASE_URL</code> → внутренний vLLM с батчингом	нет
SQLite стал узким местом	<code>DB_URL</code> → PostgreSQL; SQLAlchemy уже async	нет
нужна распределённая очередь	таблица <code>Job</code> уже есть — заменить <code>asyncio.Queue</code> на брокер	локально в <code>app/queue.py</code>

Почему `Job` — таблица, а не только очередь в памяти: не ради распределённости, а ради наблюдаемости. После перезапуска видно, какие работы остались непроверенными, и они возвращаются в очередь автоматически (`_requeue_orphans`).

6. Смена LLM-провайдера

Требование: смена провайдера не должна требовать правок кода.

Единственное место в проекте, знающее про модель, — `app/llm/client.py` . Единственное место, знающее про Ollama, — `scripts/ollama_setup.ps1` . Весь код работает через OpenAI SDK против OpenAI-совместимого API.

```
# configs/providers/ollama.env - локально, офлайн (текущий)
LLM_BASE_URL=http://localhost:11434/v1
LLM_MODEL=avito-reviewer
LLM_REASONING_EFFORT=none

# configs/providers/vllm.env - свой сервер в контуре заказчика
LLM_BASE_URL=http://vllm.internal:8000/v1
LLM_MODEL=Qwen/Qwen3.5-9B-Instruct

# configs/providers/openai.env - облако (данные покидают контур!)
LLM_BASE_URL=https://api.openai.com/v1
LLM_MODEL=gpt-4o-mini
LLM_REASONING_EFFORT=          # OpenAI не знает значение "none"
ALLOWED_HOSTS=localhost,127.0.0.1,api.openai.com
```

Профиль облака не заработает, пока его хост не добавлен в `ALLOWED_HOSTS` осознанно — офлайн-контур не даёт утечь случайно.

7. Что потребует настоящей доработки

Честный список — граница масштабируемости этого решения.

Ограничение	Когда упрёмся	Что делать
Схемы и изображения не анализируются	курсы, где основной артефакт — диаграмма (системный дизайн)	мультимодальная модель; интерфейс уже помечает «изображение не оценено»
Аутентификации нет	любой пилот с реальными студентами	SSO организации

Ограничение	Когда упрёмся	Что делать
Работа целиком в контексте	работы длиннее ~25 тыс. токенов	map-reduce по разделам; предохранитель по длине уже есть
Кластеризация типовых ошибок частотная	тонкий анализ пробелов	эмбединги и кластеризация
Бенчмарк на трёх работах	оценка качества как статистики	разметка ревьюерами на десятках работ

Ни одно из ограничений не требует переработки архитектуры — все локальны.

8. Демонстрация масштабируемости на защите

Показывается за две минуты, без правок кода:

1. Методист создаёт задание второго курса и загружает **условие бизнес-модели**.
2. Рубрика извлекается: другие критерии, другие максимумы, другие формальные требования.
3. Загружаются решения в `.xlsx` — читаются структурно, с формулами.
4. Запускается ревью: те же пять шагов пайплайна, та же проверка цитат, та же подсветка.

Тот же код, другой курс, другой формат файлов, ноль правок.

Проверено на третьем курсе — и что это стоило

Утверждение проверено не рассуждением, а прогоном. Новый методист (заведён через интерфейс существующим методистом) завёл ревьюера, трёх студентов и задание по **техническому QA**, загрузил условие, утвердил рубрику, залил три решения в `.xlsx`, распределил и получил ревью:

Работа	Балл	Цитат подтверждено	Время
слабое	12,5 / 20	84 %	60 с
среднее	15,5 / 20	96 %	115 с
хорошее	17,3 / 20	71 %	78 с

Все три пары упорядочены верно — на этом курсе система различает и среднюю работу с хорошей.

Честная часть: **правок кода это всё-таки потребовало — но не в архитектуре, а в двух общих слоях, где были зашиты предположения об одном курсе.**

1. Читатель `.xlsx` делал из листа один блок `§N`. На решении по QA лист занимал 19 тысяч символов — проверка цитат в блоке такого размера находит почти любую строку, то есть не проверяет ничего. Лист теперь режется по строкам. Эффект измерим: до правки порядок работ был нарушен (13,5 / 15,5 / 15,0 — средняя выше хорошей), после стал верным.
2. Регулярка максимумов баллов знала только запись «0–4 балла» из условия по фроду. Условие по QA пишет «(8 баллов)», и кросс-проверка суммы молча выключалась: рубрика с ошибкой модели (21 балл при заявленных в условии 20) выглядела нормальной.

Ни одна из правок не добавила ветки «если курс QA»: обобщились сами слои. Но называть это «ноль правок» было бы неточно, и мы этого не делаем.

Конфигурация

docs/configuration.md

Всё поведение задаётся `.env` и настройками в интерфейсе. Правка кода не требуется ни для смены провайдера, ни для настройки формул и порогов.

1. Переменные окружения

Файл `.env` создаётся из `.env.example`. Полное описание — `app/config.py`.

LLM

Переменная	По умолчанию	Смысл
LLM_BASE_URL	http://localhost:11434/v1	базовый URL OpenAI-совместимого API. Для Ollama суффикс <code>/v1</code> обязателен
LLM_API_KEY	ollama	Ollama ключ не проверяет, но SDK требует непустую строку
LLM_MODEL	avito-reviewer	производная модель из <code>scripts/Modelfile.reviewer</code> . Базовую <code>qwen3.5:9b</code> указывать нельзя: у неё <code>num_ctx=4096</code> , и хвост работы молча теряется
LLM_MODEL_FAST	avito-reviewer-fast	younger-модель для черновых прогонов
LLM_TEMPERATURE	0	воспроизводимость ревью
LLM_SEED	42	то же
LLM_TIMEOUT_SECONDS	180	таймаут одного вызова
LLM_MAX_RETRIES	2	число ремонтных ретраев при невалидном JSON
LLM_NUM_CTX	32768	ожидаемое окно контекста; smoke-тест сверяет с фактическим через <code>/api/show</code>
LLM_MAX_TOKENS	3072	предохранитель от разгона генерации
LLM_REASONING_EFFORT	none	критично для qwen3.5 . При включённых рассуждениях весь вывод уходит в поле <code>thinking</code> , <code>content</code> остаётся пустым, а генерация не завершается. Пустая строка — параметр не отправляется (для провайдеров, не знающих значение <code>none</code>)

Офлайн-контур

Переменная	По умолчанию	Смысл
ALLOWED_HOSTS	localhost, 127.0.0.1	строка через запятую. Всё остальное блокируется и пишется в аудит
OFFLINE_ENFORCE	true	<code>true</code> — попытка выйти наружу роняет вызов ; <code>false</code> — только запись в журнал

Персональные данные

Переменная	По умолчанию	Смысл
PII_ENABLE	true	псевдонимизация до вызова модели. Выключать только для отладки
PII_USE_NER	true	NER natasha поверх регулярок. Без неё работают только регулярки, и это явно сообщается

Приложение

Переменная	По умолчанию	Смысл
APP_HOST / APP_PORT	127.0.0.1 / 8000	адрес uvicorn
DB_URL	sqlite+aiosqlite:///./data/app.db	SQLAlchemy async. Для PostgreSQL достаточно сменить строку
DATA_DIR / UPLOAD_DIR	./data , ./data/uploads	относительные пути считаются от корня проекта, а не от cwd : иначе запуск из другой директории создаёт вторую базу
REVIEW_WORKERS	1	воркеров очереди. Больше 1 на одной GPU смысла не имеет — Ollama сериализует запросы
LOG_LEVEL	INFO	

2. Профили провайдеров

Готовые наборы в configs/providers/. Применение — копирование значений в .env .

Профиль	Назначение	Данные покидают контур
ollama.env	локальная Ollama, основной режим	нет
vllm.env	собственный vLLM в контуре заказчика — путь на прод	нет
openai.env	облачный OpenAI, доказательство провайдеро-независимости	да

Профиль облака не заработает, пока api.openai.com не добавлен в ALLOWED_HOSTS осознанно. Случайная утечка через смену модели невозможна.

3. Параметры модели: Modelfile

Два параметра нельзя задать из кода, потому что OpenAI-совместимый /v1 Ollama не принимает поле options . Они зашиты в производную модель — scripts/Modelfile.reviewer :

```
FROM qwen3.5:9b

PARAMETER num_ctx 32768      # умолчание Ollama 4096: хвост работы терялся молча
PARAMETER temperature 0
PARAMETER top_p 1
PARAMETER seed 42

# qwen3.5 приносит из базового Modelfile presence_penalty 1.5 и top_k 20.
# При temperature=0 выбор жадный, но штрафы всё равно правят logits до argmax,
# а структурный JSON состоит из повторяющихся токенов. Гасим оба.
PARAMETER presence_penalty 0
PARAMETER frequency_penalty 0
PARAMETER repeat_penalty 1.0
PARAMETER top_k 0
```

Создание и проверка — `scripts/ollama_setup.ps1`. Скрипт **ассертит фактические** значения через `/api/show`, а не то, что записано в `.env`: иначе ошибка обнаружилась бы только на демонстрации.

4. Формула балла и индекс приоритета

Настраивается в коде через `ScoringConfig` и правится из интерфейса. Считаются **два независимых числа** — `app/scoring/formula.py`.

(а) Предварительный балл

балл = $\sum(\text{вес}_i \times \text{балл}_i)$ - штрафы

Параметр	По умолчанию	Откуда
максимум критерия	1 / 4 / 2 / 2 / 1	из условия задания
вес критерия	1.0	редактируется в рубрике
штраф за просрочку	-1 балл	правило из условия
балл после жёсткого срока	0	правило из условия
штраф за нарушения оформления	выключен, 0.5 за нарушение, максимум 1.0	тумблер <code>formal_penalty_on</code>

Штраф за оформление выключен по умолчанию намеренно: условие задания его не предусматривает, а критерий «Качество оформления» уже оценивается отдельно. Включать его — решение методиста.

(б) Индекс приоритета ручной проверки

Каждый компонент — **вес плюс тумблер**, правятся ползунками в разделе «Формула приоритета» у методиста (`GET/PUT /api/scoring`). Правка сразу пересчитывает уже готовые ревью: методист двигает вес и видит, как переупорядочилась очередь ревьюеров, а не ждёт нового прогона модели.

Компонент	Вес	Что означает
<code>ai_signal</code>	0.35	признаки генеративного ИИ
<code>similarity</code>	0.25	схожесть с другой работой потока
<code>unverified_evidence</code>	0.20	цитаты не подтвердились кодом

Компонент	Вес	Что означает
<code>formal_violations</code>	0.10	формальные нарушения
<code>borderline_score</code>	0.10	балл у границы между оценками
<code>failed_steps</code>	0.30	шаг пайплайна не выполнен

Пресеты формулы

Набор весов и тумблеров сохраняется под именем и применяется к потоку одним нажатием (`GET/POST/DELETE /api/scoring/presets` , `POST .../apply`). Пресеты лежат в базе, а не в памяти процесса: настройку «строгий поток» нужно уметь применить завтра и на другом задании.

Пресет	Что делает
Базовый	значения по умолчанию из таблицы выше; точка возврата после кручения ползунков
Строгий к автоматике	поднимает вес всех сигналов недоверия: сигнал генИИ 0.5, непроверенные цитаты 0.35, схожесть 0.35, упавшие шаги 0.4
Только формальные признаки	выключает генИИ и схожесть, оставляет проверяемое кодом: формальные нарушения 0.4, непроверенные цитаты 0.3

Пресет задаёт конфигурацию **целиком**, а не поверх текущей: иначе после переключения в настройках оставались бы хвосты предыдущего, и одно и то же имя давало бы разный поток. Встроенные три удалить нельзя.

Ограничение кейса соблюдается при любом пресете: балл не меняется ни от одного веса. Проверено живьём — «Только формальные признаки» гасит вклад детектора, приоритет работы падает с 0,108 до 0,000, балл остаётся 8,0.

Тумблер детектора генИИ гасит его вклад в приоритет проверки и в рекомендацию, но **не в балл**: в балл он не входит ни во включённом, ни в выключенном состоянии. Кейс запрещает генеративной проверке автоматически влиять на оценку, и это обеспечено структурно — числа посчитаны разными формулами из разных слагаемых.

5. Веса распределения

`AllocationWeights` — `app/workload/allocator.py`.

Компонент	Вес	Смысл
<code>competence</code>	100	несоответствие компетенции направлению. Не запрет: при нехватке компетентных работа всё равно распределяется, но выбирается последней
<code>load_balance</code>	30	загрузка относительно ёмкости, в квадрате — так выравнивание сильнее штрафует перекос
<code>urgency</code>	15	горящий срок не должен попасть в конец длинной очереди
<code>repeat_continuity</code>	-25	повторное ревью выгодно оставить прежнему ревьюеру
<code>effort</code>	10	трудоемкость работы по объёму текста

6. Формальные требования

Приходят **из рубрики**, а не из кода. По умолчанию — значения ДЗ «Карта рисков продукта»:

Требование	Значение	Извлекается
объём	не более 3 страниц	автоматически
шрифт текста	Arial 11 пт	автоматически
шрифт таблиц	Arial 10 пт	автоматически
межстрочный интервал	1,15–1,5	автоматически
формат файла	DOCX, PDF	из условия
история изменений	требуется	из условия
декларация об использовании ИИ	требуется	из условия

Отдельно — `tolerated_fonts`: `Cambria Math`, `Symbol`, `Wingdings`. Word подставляет их сам (в частности, `Cambria Math` появляется в любой формуле из редактора формул), и штрафовать за это студента неверно.

7. Сроки

Параметр	По умолчанию	Смысл
<code>due_at</code>	из условия	мягкий дедлайн — срок сдачи
<code>hard_due_at</code>	<code>due_at</code> + 24 ч	после него 0 баллов
<code>review_due_at</code>	жёсткий срок + 7 дней	срок проверки
<code>due_soon_lead_s</code>	3600	за сколько до срока показывать «скоро дедлайн»

Системные часы не подкручиваются. Время всегда реальное, управляются сами сроки. Для демонстрации есть пресет «10 с / 40 с»: тот же механизм обслуживает и реальные сроки в днях, и десятисекундные на защите.

8. Настройки в интерфейсе

Что меняется без правки файлов:

Где	Что
Шапка → «+ Новое задание»	создание задания: название, направление, номер ДЗ, канал сдачи
Вкладка методиста → Рубрика	критерии, максимумы, веса, формальные требования; загрузка условия; ручной JSON-редактор
Вкладка методиста → Загрузка работ	загрузка файлов студентов (docx, pdf, xlsx, md, txt, ipynb, исходный код, zip с репозиторием); повторная сдача как новая версия существующей работы
Вкладка методиста → Участники	добавление ревьюеров с ёмкостью и компетенциями, студентов, методистов; отключение
Вкладка методиста → Формула приоритета	веса и тумблеры всех компонентов, штраф за оформление, пресеты; пересчёт готовых ревью на месте
Вкладка методиста → Аналитика → Профиль студента	динамика баллов и радар по критериям любого студента
Вкладка «Качество»	результаты бенчмарка: ранжирование, разброс, доля подтверждённых цитат, разбор по парам
Вкладка «Приватность»	счётчики вызовов, журнал обращений, что нашёл ПДн-щит
Вкладка методиста → Сроки	три срока с точностью до секунды, пресеты, режим демонстрации

Где	Что
Вкладка методиста → Распределение	автораспределение, ручной перенос работы
Вкладка методиста → Работы	выгрузка XLSX, CSV и JSON; повторный запуск ревью по любой работе
Вкладка ревьюера	баллы по критериям, текст фидбека, вердикт по сигналу ИИ; четыре слоя подсветки документа; сравнение с предыдущей версией; выгрузка MD / HTML / JSON
Шапка	тёмная и светлая тема

Честные ограничения

docs/limitations.md

Список того, чего решение **не** делает. Полезнее умолчания: ревьюер должен знать, где кончается автоматическая проверка, а жюри — где кончается наше утверждение.

Каждое ограничение из первого раздела видно **в самом интерфейсе**, а не только здесь.

1. Ограничения проверки, видимые пользователю

Схемы и изображения не анализируются

Текстовая модель не видит картинки. Схемы CJM и матрицы «Вероятность × Влияние» часто нарисованы изображением.

Что делает система: каждое изображение попадает в отчёт с пометкой «изображение не оценено», а критерий, опирающийся на схему, получает запись в `gaps`. Молча оценивать наугад — недопустимо.

Насколько это существенно: у двух из трёх реальных работ есть `word/media/image1.png`, и именно поэтому критерий «Обоснованная приоритизация» у всех трёх получает 1,5 из 2 — визуальную матрицу проверить нечем. Это не дефект модели, а граница метода.

Метаданные документа доступны не всегда

У экспорта из Google Docs `docProps/app.xml` отсутствует целиком. Недоступны время редактирования, число слов и страниц.

Что делает система: сигнал генИИ по метаданным помечается «недоступен» — это **не то же самое**, что «признаков не найдено». Объём оценивается по числу символов и явно помечается как оценка. Уверенность итогового сигнала снижается, потому что часть ансамбля отсутствует.

Проверено: у слабого решения `app.xml` нет — это реальный экспорт из Google Docs.

Межстрочный интервал может быть не определён

Значение наследуется по цепочке: параграф → стиль → `docDefaults`. Если его нет нигде, Word применяет собственное умолчание, и утверждать, каким оно окажется при открытии, мы не можем.

Что делает система: отвечает «не определено» и просит проверить визуально. Трактовать отсутствие данных как нарушение значило бы оштрафовать корректную работу.

Проверено: у хорошего решения интервал не задан ни в одном из 60 блоков.

История изменений по локальной копии не проверяется

Условие требует доступ на редактирование с видимой историей. История живёт в сервисе (Google Docs, Word Online), а не в скачанном файле.

Что делает система: ищет косвенные признаки — правки в режиме рецензирования, примечания, номер редакции, соотношение дат создания и изменения. Проверка **никогда не выносит «нарушение»**, только «соблюдено» или «не определено»: по локальной копии отсутствие истории не доказывается.

Требования проверяются только там, где они заданы условием

Формальный слой не имеет значений по умолчанию. Если условие не задаёт объём, шрифт, кегль, интервал или список форматов, соответствующая проверка возвращает статус «неприменимо» с пояснением «в условии задания не задано требование».

Это не пробел, а исправленный дефект. Пороги ДЗ по продуктовому фроду стояли умолчаниями, и каждый другой курс молча получал чужие требования: работа по системному дизайну, которую её условие разрешает сдавать в Markdown, теряла балл за «формат MD не входит в разрешённые (DOCX, PDF)».

Следствие для новых курсов. Условие, не описывающее оформление, получит пять проверок со статусом «неприменимо». Это правильное поведение, но означает, что формальный слой на таком курсе почти не даёт сигнала — и вес проверки оформления в рубрике стоит задавать осознанно.

Детектор по логитам не внедрён — и это измеренное решение

Мы попробовали добавить пятый сигнал: насколько текст предсказуем для самой модели. Идея та же, что у DetectGPT — машинный текст лежит в области высокой вероятности породившей его модели.

Что получилось. Прямой путь закрыт: Ollama отдаёт логпробы только для сгенерированных токенов, а не для заданного текста. Обходной приём нашёлся на стандартных полях OpenAI — префикс кладётся в сообщение роли `assistant`, и модель дописывает начатое. Первый замер выглядел отлично:

текст	доля угаданных токенов
порождён этой же моделью	56 %
человек, слабое решение	22 %
человек, среднее решение	31 %
человек, хорошее решение	25 %

Почему не внедрили. Проверка устойчивости отменила результат. При сдвиге точек замера по тому же тексту значения гуляют на 15–20 пунктов:

текст	сдвиг 0	0,25	0,5	0,75
порождён моделью	42 %	42 %	50 %	32 %
человек, слабое	22 %	42 %	32 %	22 %
человек, хорошее	32 %	40 %	25 %	38 %

Разброс равен самой разнице между машиной и человеком, а диапазоны пересекаются полностью. Первые 56 % против 22 % были везением выборки.

Чтобы снять шум, нужны сотни замеров вместо сорока — это минуты на работу против восьми секунд. Плата несоразмерна: сигнал по правилам кейса всё равно не может влиять на оценку, а к четырём имеющимся добавил бы шум под видом доказательства. Код удалён, замеры сохранены здесь.

Отдельное ограничение метода, которое осталось бы даже при устойчивом сигнале: он узнаёт прежде всего **свою** модель. Текст, написанный другой моделью, предсказуем для локальной хуже.

Детектор генеративного ИИ вероятностный

Не является доказательством нарушения и не влияет на балл автоматически.

Что делает система: показывает скор, уверенность, основания и **ограничения метода**; вклад идёт только в индекс приоритета ручной проверки; ревьюер подтверждает или отклоняет сигнал, и его решение фиксируется.

Метод в принципе не отличает текст, написанный с помощью ИИ и затем переработанный студентом, от полностью самостоятельного. Аккуратный деловой стиль сам по себе шаблонен, поэтому шаблонность — слабый признак.

Работа может не влезть в один проход модели

Что не работает: окно контекста конечно. Решения в `.xlsx` из набора примеров кейса доходят до 148 тысяч символов при бюджете около 45 тысяч.

Что делает система: обрезает работу **сама**, по границам блоков, с пометкой прямо в тексте и предупреждением в карточке ревьюера. Без этого промпт молча укорачивал сам сервер модели, и балл выставлялся по случайному куску работы — при этом ревьюер видел обычный результат.

Чего это стоит: хвост длинной работы в автоматическую оценку не попадает и требует ручного просмотра. Правильное решение — map-reduce по разделам; оно за границами MVP.

Скан без текстового слоя не проверяется

Что не работает: из PDF, собранного из картинок, текст не извлекается. В наборе примеров такое решение есть — 1,3 МБ, одна страница, ноль символов.

Что делает система: отвергает работу на первом же шаге пайплайна и ставит максимальный приоритет ручной проверки. Балл не выставляется вовсе: оценить документ, которого модель не видела, хуже, чем не оценить.

Чего не хватает: OCR. Он добавил бы зависимость и время, а случай редкий — правильнее попросить студента сдать текстовый документ.

Таблицы, склеенные в крупные блоки, хуже цитируются

Подтверждаемость цитат различается по работам: 81 % у слабой против 92 % у хорошей. Причина не в модели: таблицы там образуют крупные блоки, и модель цитирует их с переформатированием, из-за чего нечёткое сопоставление не находит точного вхождения.

Что делает система: честно сообщает «цитаты не подтверждены» и поднимает приоритет ручной проверки. Балл при этом **не снижается** — техническая неудача сопоставления не доказывает невыполнение критерия.

Крайний случай этого же дефекта нашёлся на курсе QA и исправлен: лист `.xlsx` целиком становился одним блоком на 19 тысяч символов. В блоке такого размера нечёткий поиск находит почти любую строку — проверка цитат превращалась в формальность. Теперь лист режется по строкам с повтором шапки, и ранжирование на том курсе стало верным.

2. Ограничения оценки качества

- **Бенчмарк на трёх работах одного задания** — показатель направления, а не статистика. Для оценки качества как метрики нужна разметка ревьюерами на десятках работ.
- **Эталонных баллов преподавателя нет.** Репозиторий кейса задаёт только относительные уровни, поэтому измеряется правильность **порядка**, а не попадание в число.
- **Метрика «согласие ревьюера с AI»** реализована и считается, но на трёх работах статистически не значима. Как результат не заявляется.
- **Базовая линия ручных действий** выведена из описания процесса в кейсе, а не измерена секундомером у ревьюеров. Расчёт приведён пошагово в `as-is-to-be.md`, чтобы его можно было оспорить.

3. Ограничения реализации

Ограничение	Почему допустимо в MVP	Что нужно для пилота
Вход есть, полноценной аутентификации нет. Логин и пароль проверяются, пароль хранится хешем <code>pbkdf2</code> , но сессия — это по-прежнему идентификатор в заголовке: нет подписанного токена, срока жизни сессии, блокировки после подбора и восстановления пароля	форма входа отвечает на вопрос «кто вы» и делает роли осязаемыми; контур защищён не паролем, а тем, что приложение не выходит в сеть	SSO организации, подписанные сессии со сроком, ограничение частоты попыток
Одна GPU, один воркер	Ollama всё равно сериализует запросы; больше воркеров только раздули бы очередь	vLLM с батчингом
SQLite	ноль инфраструктуры, поток одного курса	PostgreSQL — правка <code>DB_URL</code> , кода нет
Результаты ревью в JSON-колонках	структура эволюционирует, миграции на этом сроке дороже гибкости	нормализация при стабилизации схемы
Журнал вызовов в памяти	демо — один процесс	таблица с ротацией
Уведомления только внутри приложения	контур офлайн; Telegram и SMTP потребовали бы выхода в сеть	внешние каналы после согласования контура
Нет разграничения прав внутри роли	один поток, три ревьюера	права на уровне программы и курса
Кластеризация типовых ошибок частотная	для «что объяснить курсу подробнее» хватает частотного списка	эмбединги и кластеризация
Работа целиком в контексте	реальные работы 3–9 тыс. символов, укладываются с запасом	map-reduce по разделам; предохранитель по длине уже есть

4. Что сознательно не входит в скоуп

Границы заданы кейсом, и мы их не расширяли:

- поддержка всех программ, типов заданий и каналов сдачи одновременно — достаточно одного задания и одного канала;
- автоматическое выставление итоговой оценки;
- автоматическое применение санкций по признакам генеративного ИИ;
- полноценное масштабирование на все курсы — достаточно показать возможность адаптации без переработки архитектуры;
- передача персональных данных, внутренних материалов или кода в незащищённые сервисы.

5. Отклонения от первоначального плана

Прозрачности ради — что изменилось по ходу работы и почему.

План	Факт	Причина
APScheduler для точных триггеров сроков	асинхронный цикл раз в секунду	приложение уже живёт в event loop; тик раз в секунду ничего не стоит, а зависимость и её конфигурация обошлись бы дороже. Точность перехода — та же секунда
Отдельные модули app/api/*.py на каждый ресурс	один app/api/routes.py с разделами	маршрутов немного; разнесение по десятку файлов на этом объёме добавило бы навигации больше, чем ясности
Recharts для графиков	рукописный инлайновый SVG	гистограмма, столбцы по дням, линия динамики и радар по критериям не стоят зависимости, а контур офлайн — CDN недоступен
Файловая структура из ~60 модулей	48 файлов: agent/steps/* → pipeline.py, scoring/presets.py и penalties.py → formula.py, privacy/redactor.py → pii.py, notify/rules.py и inbox.py → scheduler.py и deadlines.py	интерфейсы те же, файлов меньше; дробление на этом объёме добавляло бы навигации, а не ясности

Отклонения зафиксированы здесь, а не сглажены в описании: план — рабочий документ, а не обещание.

Все пятнадцать механизмов плана реализованы, включая те, что были доделаны последними: слой персональных данных в просмотрщике, повторная сдача с версиями и сравнением по критериям, пресеты формулы приоритета, страница «Качество» с результатами бенчмарка, профиль студента с динамикой и радаром, график сдач по дням, сводка методисту и выгрузка ревью в JSON.

Сценарий демонстрации

docs/demo_script.md

Пошаговое руководство со скриншотами — `guide.md`. Здесь — тайминг и трассируемость требований.

План рассказа и показа. Таблица трассируемости внизу: строка на каждое требование кейса — где реализовано, как показать, что сказать.

Подготовка (до выхода на защиту)

```
# 1. Убедиться, что модель настроена и параметры фактически применены
powershell -ExecutionPolicy Bypass -File scripts\ollama_setup.ps1

# 2. Пересоздать демо-данные (рубрика останется НЕутверждённой – так надо)
python -m app.seed --reset --no-extract

# 3. Запустить
powershell -ExecutionPolicy Bypass -File scripts\run.ps1
```

Три вкладки браузера на `http://127.0.0.1:8000`, вход по логину и паролю:

Вкладка	Логин	Пароль	Роль
1	sokolova.i	avito2026	методист
2	dyachenko.o	avito2026	ревьюер
3	lebedeva.a	avito2026	студент

Логины и пароль напечатаны на самом экране входа — списком по ролям, кнопка «Демонстрационные доступы». Печатать вручную не придётся.

Сессия хранится в `sessionStorage` — он изолирован по вкладке, поэтому три роли живут одновременно в одном браузере, без режима инкогнито.

Проверить перед началом: в шапке горит зелёное «внешних вызовов: 0» и «модель локально». Если нет — модель не поднялась.

Тайминг: 8 минут

Мин	Что показываем	Что говорим
0:00–0:40	Шапка, три вкладки	«Три роли, одна система. Вверху — счётчик внешних вызовов: ноль. Это не декларация, а число из аудита: контур офлайн, единственный адрес — локальная модель.»
0:40–1:40	Методист: загрузка условия → рубрика	«Загружаю условие. Пять критериев, максимумы 1-4-2-2-1, объём, шрифт, кегль, интервал, срок и правило штрафа извлечены автоматически. Числа берёт код регулярками, названия — модель: цифры в условии однозначны, доверять здесь модели незачем.»
1:40–2:00	Нажать «Утвердить рубрику»	«Это черновик. Ревью по неутверждённой рубрике не запускается — ошибка в критериях исказила бы все баллы разом. Утверждает человек.»

Мин	Что показываем	Что говорим
2:00–2:40	Распределение → график до/после	«Павел Кузнецов уже загружен на три работы из четырёх, Ольга свободна, Тимур не проверяет продуктовый фронт. Венгерский алгоритм: все три работы уходят Ольге Дьяченко. Это точный оптимум задачи о назначении, а не «раздать поровну».»
2:40–3:10	«Проверить все» → прогресс по SSE	«Очередь пошла. Прогресс приходит по событиям, без перезагрузок. Одна работа — около 45 секунд, семь вызовов модели.»
3:10–5:10	Ревьюер: карточка работы	см. разбор ниже
5:10–5:50	Сроки: режим демонстрации	см. разбор ниже
5:50–6:20	Ревьюер правит балл и подтверждает	«Правлю балл, правлю текст. Итог публикуется студенту только сейчас — по нажатию человека.»
6:20–6:50	Студент: мгновенно видит фидбек	«Обратная связь пришла сразу. Балл, штраф, что улучшить. Внутренних флагов студент не видит — сервер их этой роли не отдаёт.»
6:50–7:40	Методист: аналитика, схожесть, выгрузка	«Воронка, распределение баллов, загрузка ревьюеров, типовые ошибки потока. Ручных действий было 11 на работу, стало 3 — число считается по журналу фактических действий. Выгрузка XLSX заменяет ручной перенос в таблицу.»
7:30–7:50	Вкладка «Качество»	«Цифры бенчмарка прямо здесь. Ранжирование 67 %: слабую работу система отделяет от обеих сильных устойчиво, среднюю и хорошую — нет, разрыв 0,3 балла, и он весь в одном критерии из пяти. Мы это показываем, а не прячем: подгонять веса под три примера значило бы настроиться на выборку.»
7:50–8:00	Вкладка «Приватность»	«Финальный кадр. Внешних вызовов — ноль, и это число из журнала перехваченных обращений, а не из настроек. Здесь же видно, что нашёл ПДн-щит и что маскирование идёт до вызова модели.»

Разбор ключевого экрана: карточка ревьюера (2 минуты)

Порядок показа сверху вниз — так же, как читает ревьюер.

1. Очередь отсортирована по приоритету.

«Сверху не самая слабая работа, а та, где автоматика вероятнее ошиблась. Индекс приоритета — отдельное число, считается по флагам риска.»

2. Формальные проверки — четыре статуса, не два.

«Слой без модели: объём, шрифт, кегль, интервал. У этой работы нарушение в шрифте таблиц. А вот здесь — «не определено»: межстрочный интервал не задан ни в параграфах, ни в стилях. Мы не пишем «нарушение», потому что данных нет. Это отдельный статус специально: трактовать отсутствие данных как нарушение значило бы оштрафовать корректную работу.»

3. Баллы по критериям с проверенными цитатами. ← *главный аргумент*

«Каждый балл подкреплён дословной цитатой с номером блока. Галочка означает, что **код** нашёл эту цитату в указанном блоке нечётким сравнением. Вот здесь крестик — цитата не подтвердилась. Балл при этом не снижен: неудача сопоставления не доказывает невыполнение критерия. Но критерий помечен, и приоритет ручной проверки вырос.»

Кликнуть по цитате → скролл к фрагменту в документе с подсветкой.

«Модель физически не может объявить свою проверку пройденной: схема, которую она заполняет, содержит только номер блока и текст цитаты. Статус проставляет код.»

4. Сигнал генеративного ИИ.

«Четыре независимых сигнала: стилометрия, форматные артефакты, метаданные, суждение модели. Здесь метаданные помечены «недоступен» — у этой работы нет `docProps/app.xml`, это экспорт из Google Docs. Не «признаков нет», а «проверить нечем». Из-за неполноты ансамбля уверенность снижена.

Справа — ограничения метода. Сигнал не является доказательством, на балл не влияет. Отклоняю его — решение зафиксировано за мной.»

Нажать «Отклонить сигнал».

5. Трасса выполнения.

«Что система успела проверить и сколько это заняло. Если бы шаг упал, он был бы здесь с крестиком, а ревью всё равно дошло бы до конца.»

Разбор момента со сроками (40 секунд)

Переключиться на вкладку методиста, нажать **«Режим демонстрации: 10 с / 40 с»**.

«Системные часы я не трогаю — время настоящее. Двигаю сами сроки: мягкий дедлайн через 10 секунд, жёсткий через 40.»

Смотреть на состояние работы:

Момент	Что происходит	Что сказать
~5 с	жёлтое «сорок дедлайн», пинг студенту	«Уведомление ушло студенту по событию.»
~10 с	оранжевое «просрочено, штраф –1», балл пересчитан	«Правило из условия: досдача в течение суток стоит –1 балл. Балл пересчитан автоматически, ревьюер видит новое число.»
~40 с	красное «просрочено окончательно, 0 баллов»	«После жёсткого срока — ноль. Тоже правило из условия задания.»

Важная оговорка (если спросят): работа, сданная **до** дедлайна, не станет просроченной, если срок сдвинуть вперёд. Состояние сданной работы определяется моментом сдачи. Чтобы показать штраф на уже сданных работах, нужно сдвинуть срок **в прошлое** — это отдельный, тоже реальный сценарий («настоящий дедлайн был раньше»).

Повторная сдача — как показать

В демо-данных второй версии нет намеренно: файла доработанной работы среди выданных примеров не выдано, а подставить вместо него чужое решение нельзя — детектор схожести честно покажет 100 %, и получится ложная история о списывании. Показывается вживую за двадцать секунд:

1. Методист → «Загрузка работ» → студент **Анна Лебедева**;
2. в поле «Повторная сдача» выбрать её текущую работу;
3. загрузить исправленный файл.

Что происходит: создаётся версия 2, предыдущая уходит из очереди ревьюера, но остаётся в истории студента и доступна для сравнения; ревьюер сохраняется и получает уведомление с прошлым баллом. После прогона ревью в карточке появляется таблица «было / стало» по каждому критерию — видно, что именно студент доработал, а не только сдвинулся ли общий балл.

Устаревшие версии исключены и из очереди, и из отчёта о схожести: работа неизбежно похожа на собственную предыдущую версию, и в отчёте о плагиате такая пара была бы ложным обвинением.

Запасной вариант при сбое

Что сломалось	Что делать
Модель не отвечает	Данные уже в базе: все три ревью выполнены заранее . Показывать карточки как есть, ревью на защите не запускать. Сказать: «Ревью выполнено до демонстрации, чтобы не тратить ваше время на ожидание.»
Интерфейс не открылся	<code>python -m app.cli review <работа> -с <условие></code> — то же ревью в консоли, с цветным выводом
UnauthorizedAccess при запуске скрипта	Политика выполнения Windows PowerShell 5.1 — Restricted . Добавить -ExecutionPolicy Bypass (действует только на процесс)
SSE отвалился	Обновить страницу; данные приходят и обычными запросами
Ollama не запущена	<code>powershell -ExecutionPolicy Bypass -File scripts\ollama_setup.ps1</code> — поднимет сервер и проверит параметры
Нужно повторить сценарий	«Сбросить демо» у методиста → <code>python -m app.seed --reset</code>

Правило: ничего не пересчитывать на защите без необходимости. Данные предзагружены, демонстрация не требует работы модели.

Заготовленные ответы на вероятные вопросы

«Модель может выдумать цитату — как вы это ловите?»

Документ режется на пронумерованные блоки. Модель обязана указать номер блока и дословный текст. Код ищет цитату в указанном блоке нечётким сравнением с порогом 82 %. Не нашлась — критерий помечен «не подтверждён», приоритет ручной проверки вырос, балл не изменён. Есть тест с выдуманной правдоподобной цитатой — она отбраковывается.

«Почему нечёткое сравнение, а не точное?»

Модель нормализует пробелы, меняет регистр и кавычки при переносе. Посимвольное равенство отбраковывало бы честные цитаты. Порог подобран так, чтобы пережить нормализацию, но отбраковать пересказ.

«Покажите, что тумблер детектора не меняет балл»

Открываю «Формулу приоритета», выключаю тумблер «Признаки генеративного ИИ». Приоритет упал с 0,11 до 0,00, балл остался 8,0 — пересчёт применился к готовым ревью сразу. Это же проверяется тестом `test_ai_signal_never_affects_score` на трёх положениях тумблера.

«Почему сигнал ИИ не влияет на балл?»

Это запрещено условиями кейса, и у нас это обеспечено структурно, а не настройкой: считаются два независимых числа разными формулами. Сигнал входит только в индекс приоритета проверки. Тумблер детектора гасит его вклад в приоритет; в балл он не входит ни в одном состоянии.

«Насколько это воспроизводимо?»

temperature=0, фиксированный seed, погашенные штрафы сэмплирования. Разброс баллов на трёх прогонах — ноль. Проверено и в CLI, и через приложение: одна работа даёт один и тот же балл.

«Какое качество на реальных работах?»

Слабую работу система отделяет устойчиво: 6,5 против 8,3 и 8,0, разрыв 1,5–1,8 балла во всех трёх прогонах. Среднюю и хорошую не различает — разрыв 0,3, и причина известна точно: по четырём критериям из пяти у них одинаковый балл, весь разрыв даёт пятый — оформление, где критерий требует таблиц, а у хорошей работы их нет. Подтверждаемость цитат 88 %, разброс между прогонами ноль. Три работы одного задания — показатель направления, а не статистика.

«Как это масштабируется на другой курс?»

Загрузкой условия и утверждением рубрики — без единой строки кода. Могу показать на втором курсе с решениями в .xlsx : другой формат файлов, другие критерии, тот же пайплайн.

«Сколько это стоит в эксплуатации?»

Ноль переменных затрат: локальная модель, GPU уже есть. На 1000 работ облачный API уровня GPT-4o mini обошёлся бы примерно в 5,5 тыс. рублей и вынес бы персональные данные из контура, что кейс запрещает.

«А если модель ошибётся или упадёт?»

Пайплайн — список шагов. Упавший шаг пишется в трассу, ревью продолжается, критерий помечается «не оценён». Невалидный JSON лечится ремонтным ретраем. Ревьюер всегда видит, что именно система успела проверить.

«Почему нет аутентификации?»

Она вне объёма MVP, и мы сказали об этом прямо на экране входа. Форма пароля создавала бы ложное впечатление защищённости. Для пилота нужен SSO организации. При этом изоляция данных по роли реальная и сделана на сервере: студенту внутренние поля не отображаются вовсе.

Таблица трассируемости требований

Требование кейса	Где реализовано	Как показать
Анализ процесса, выбор этапов автоматизации	docs/as-is-to-be.md	слайд с таблицей 7 шагов и базовой линией 11 действий
Целевой процесс и архитектура	docs/as-is-to-be.md , docs/architecture.md	схема TO-BE
Распределение с учётом объёма и нагрузки	app/workload/allocator.py	вкладка методиста → «Распределить» → график до/после

Требование кейса	Где реализовано	Как показать
Предварительное ревью по критериям + рекомендации	app/agent/ , app/formal/	вкладка ревьюера → карточка работы
Признаки генИИ: уверенность и основания	app/detect/ai_text.py	панель сигнала: скор, уверенность, основания, ограничения, кнопки вердикта
Доп. 1: уведомления о дедлайнах	app/notify/scheduler.py	режим демонстрации 10 с / 40 с, колокольчик
Доп. 2: расчёт штрафа по правилу из условия	app/notify/deadlines.py , app/scoring/formula.py	переход в штрафную зону → балл пересчитан
Доп. 3: аналитика прогресса и качества	GET /api/analytics	вкладка методиста → «Аналитика»
Доп. 4: плагиат и дублирование	app/detect/similarity.py	«Схожесть работ» → «Сравнить»
Доп. 5: персонализированная обратная связь	шаг step_feedback	вкладка студента
Доп. 6: типовые ошибки → улучшение материалов	_common_gaps	«Типовые ошибки потока»
Доп. 7: детекция и защита ПДн	app/privacy/ , web/src/pages/Privacy.tsx	вкладка «Приватность»: счётчики, журнал вызовов, находки щита
Добавление курса без правок кода	POST /assignments , web/src/pages/Manage.tsx	«+ Новое задание» → условие → рубрика → участники → работы
Настройка формулы приоритета	GET/PUT /api/scoring , web/src/pages/Scoring.tsx	«Формула приоритета»: сдвинуть вес → очередь переупорядочилась; пресет «Строгий к автоматике» → тот же эффект одним нажатием
Повторная сдача после доработки	POST /submissions с replaces , PreviousVersion	загрузить вторую версию → у ревьюера таблица «было / стало» по критериям
Профиль студента	GET /api/students/{id}/profile	вкладка студента → «Мой прогресс»: динамика и радар по критериям
Сквозной сценарий от пула работ до фиксации	весь сценарий выше	8 минут по таймингу
Тестирование на трёх вариантах	app/bench/run_bench.py , вкладка «Качество»	вкладка «Качество»: ранжирование, разброс, доля подтверждённых цитат и неразделённая пара — прямо в интерфейсе, а не только в markdown
Обоснование подхода, стека, моделей, интеграций	docs/architecture.md , docs/cost.md	слайд стека
Инструкции агента	docs/agent_instructions.md , app/agent/prompts.py	слайд с правилами и тем, какую проблему решает каждое
Масштабирование на другие курсы	docs/scalability.md	загрузка условия второго курса живём

Требование кейса	Где реализовано	Как показать
Стоимость эксплуатации	<code>docs/cost.md</code>	таблица локально против облака
Меры защиты данных	<code>docs/security.md</code>	вкладка «Приватность»
Обработка ошибок моделей	<code>docs/agent_instructions.md</code>	трасса выполнения в карточке
Человек в итоговом решении	<code>docs/security.md</code>	«Подтвердить» и вердикт по сигналу ИИ
Ограничения	<code>docs/limitations.md</code>	«изображение не оценено», «не определено», «недоступен» в интерфейсе
Измеримые показатели эффекта	<code>GET /api/analytics → effect</code>	вкладка «Качество» → «Эффект: измеримые показатели»: ручные действия было/стало, время обработки, просроченные проверки
Вход по логину и паролю, регистрация	<code>app/auth.py</code> , <code>POST /api/login</code> , <code>POST /api/register</code>	стартовый экран: форма входа, вкладка «Регистрация»
Студент сдаёт работу сам	<code>POST /api/submissions</code> роль <code>student</code>	вкладка студента → «Сдать работу»
Работа со свободной темой	<code>new_assignment_title</code> в <code>POST /api/submissions</code>	у студента в списке заданий — «своя тема»: название задаёт он сам
Критерии текстом, без файла условия	<code>POST /api/assignments/{id}/condition/text</code>	«Критерии оценивания» → «Написать текстом»
Ревьюер заводит задание	<code>POST /api/assignments</code> роль <code>reviewer</code>	вкладка ревьюера → «+ Новое задание»; студент видит его при сдаче